



JUBE

BENCHMARKING
ENVIRONMENT

JUBE Documentation

Release 2.7.1

**2024, JUBE Developer Team,
Forschungszentrum Jülich GmbH**

Oct 11, 2024

CONTENTS

1	Introduction	1
2	JUBE tutorial	3
2.1	Installation	3
2.2	Configuration	4
2.3	Input format	4
2.4	Hello World	4
2.5	Help	7
2.6	Parameter space creation	8
2.7	Step dependencies	9
2.8	Loading files and substitution	10
2.9	Creating a result table	12
3	Advanced tutorial	17
3.1	Schema validation	17
3.2	Scripting parameter	18
3.3	Scripting pattern	20
3.4	Statistic pattern values	22
3.5	Jobsystem	24
3.6	Include external data	27
3.7	Tagging	30
3.8	Platform independent benchmarking	31
3.9	Multiple benchmarks	32
3.10	Shared operations	33
3.11	Environment handling	34
3.12	Parameter dependencies	35
3.13	Parameter update	37
3.14	Step iteration	39
3.15	Step cycle	42
3.16	Parallel workpackages	43
3.17	Result database	44
3.18	Creating a do log	47
3.19	The duplicate option	47
4	Frequently Asked Questions	51
4.1	Parameter groups	51
4.2	Workdir change	52
4.3	XML character handling	53
4.4	YAML character handling	53
4.5	Analyse multiple output files	54
4.6	Extract data from a specific text block	55
4.7	Restart a workpackage execution	56
5	Command line documentation	57
5.1	general	57

5.2	run	57
5.3	continue	58
5.4	analyse	59
5.5	result	59
5.6	output	60
5.7	comment	60
5.8	remove	61
5.9	info	61
5.10	log	61
5.11	status	62
5.12	tag	62
5.13	complete	62
5.14	help	62
5.15	update	63
6	Glossary	65
	Index	83

INTRODUCTION

Automating benchmarks is important for reproducibility and hence comparability which is the major intent when performing benchmarks. Furthermore managing different combinations of parameters is error-prone and often results in significant amounts of work especially if the parameter space gets large.

In order to alleviate these problems *JUBE* helps performing and analyzing benchmarks in a systematic way. It allows custom work flows to be able to adapt to new architectures.

For each benchmark application the benchmark data is written out in a certain format that enables *JUBE* to deduct the desired information. This data can be parsed by automatic pre- and post-processing scripts that draw information, and store it more densely for manual interpretation.

The *JUBE* benchmarking environment provides a script based framework to easily create benchmark sets, run those sets on different computer systems and evaluate the results. It is actively developed by the Jülich Supercomputing Centre of Forschungszentrum Jülich, Germany.

JUBE TUTORIAL

This tutorial is meant to give you an overview about the basic usage of *JUBE*.

2.1 Installation

Requirements: *JUBE* needs **Python 3.2** (or any higher version)

If you plan to use *YAML* based *JUBE* input files, you have to add the *pyyaml* module <https://pyyaml.org> to your *Python* module library. Additionally the *ruamel.yaml* module <https://pypi.org/project/ruamel.yaml> is optional. If installed it is used to verify the validity of the *YAML* files.

To use the *JUBE* command line tool, the `PYTHONPATH` must contain the position of the *JUBE* package. This can be achieved in different ways:

- You can use the **installation script** to copy all files to the right position (preferred):

```
>>> python setup.py install --user
```

This will install the *JUBE* package files and executables to your `$HOME/.local` directory. Instead of `--user` also a user specific `--prefix` option is available. Here you might have to set the `PYTHONPATH` environment variable first (this will be mentioned during the install process).

- You can utilize `pip[3]` to take care of the installation process (including the download)

```
>>> pip3 install http://apps.fz-juelich.de/jsc/jube/download.php?version=latest -
    ↪ -user
# or
>>> pip3 install http://apps.fz-juelich.de/jsc/jube/download.php?version=latest -
    ↪ -prefix=...
```

You might have to adjust your `PYTHONPATH`.

- You can add the **parent folder path** of the *JUBE* package-folder (`jube` directory) to the `PYTHONPATH` environment variable:

```
>>> export PYTHONPATH=<parent folder path>:$PYTHONPATH
```

- You can move the *JUBE* package by hand to an existing Python package folder like `site-packages`

To use the *JUBE* command line tool like a normal command line command you can add it to the `PATH` environment variable:

```
>>> export PATH=$HOME/.local/bin:$PATH
```

To check your final installation, you can use

```
>>> jube --version
```

which should highlight the current version number.

2.2 Configuration

The main *JUBE* configuration bases on the given input configuration file. But in addition, some shell environment variables are available which can be used to set system specific options:

- **JUBE_INCLUDE_PATH**: Can contain a list of paths (seperated by :) pointing to directories, which contain system relevant include configuration files. This technique can be used to store platform specific parameter in a platform specific directory.
- **JUBE_EXEC_SHELL**: *JUBE* normally uses `/bin/sh` to execute the given shell commands. This default shell can be changed by using this environment variable.
- **JUBE_GROUP_NAME**: *JUBE* will use the given *UNIX* groupname to share benchmarks between different users. The group must exist and the *JUBE* user must be part of this group. The given group will be the owner of new benchmark runs. By default (without setting the environment variable) all file and directory permissions are defined by the normal *UNIX* rules.
- **JUBE_VERBOSE**: The verbosity level can normally be specified using the `-v` command line option. This environment variable can be used to set a default verbose level, where `JUBE_VERBOSE=1` is equivalent to `-v`, `JUBE_VERBOSE=2` is equivalent to `-vv` and so on. If the command line option is set, the environment variable is ignored.
- **JUBE_BENCHMARK_OUTPATH**: This environment variable can be used to set a default benchmark outpath, relative to the execution directory. Alternatively, it can be set using the `--outpath` command line option or the `outpath` attribute of the benchmark tag. The command line option overrides the environment variable, which in turn overrides the attribute in the input file.

BASH autocompletion can be enabled by using the `eval "$(jube complete)"` command. You can store the command in your bash profile settings if needed.

2.3 Input format

JUBE supports two different types of input formats: *XML* based files and *YAML* based files. Both formats support the same amount of *JUBE* features and you can select your more preffered input format.

The following sections will always show all examples using both formats. However the explanations will mostly stick to the *XML* format but can be easily transfered to the *YAML* solution.

Both formats depends on a specifc special scharacter handling. More details can be found in the following FAQ sections:

- [XML character handling](#)
- [YAML character handling](#)

Internally *JUBE* always uses the *XML* based format, by converting *YAML* based configuration files into *XML* if necessary. This is why parsing error messages might point to *XML* errors even if the *YAML* format was used.

2.4 Hello World

In this example we will show you the basic structure of a *JUBE* input file and the basic command line options.

The files used for this example can be found inside `examples/hello_world`.

The input file `hello_world.xml`:

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <jube>
3   <benchmark name="hello_world" outpath="bench_run">
4     <comment>A simple hello world</comment>
5
6     <!-- Configuration -->
7     <parameterset name="hello_parameter">
```

(continues on next page)

(continued from previous page)

```

8     <parameter name="hello_str">Hello World</parameter>
9     </parameterset>
10
11     <!-- Operation -->
12     <step name="say_hello">
13         <use>hello_parameter</use> <!-- use existing parameterset -->
14         <do>echo $hello_str</do> <!-- shell command -->
15     </step>
16 </benchmark>
17 </jube>

```

The input file `hello_world.yaml`:

```

1 name: hello_world
2 outpath: bench_run
3 comment: A simple hello world
4
5 #Configuration
6 parameterset:
7     name: hello_parameter
8     parameter: {name: hello_str, _: Hello World}
9
10 #Operation
11 step:
12     name: say_hello
13     use: hello_parameter #use existing parameter
14     do: echo $hello_str #shell command

```

Every *JUBE XML* based input file starts (after the general *XML* header line) with the root tag `<jube>`. This root tag must be unique. *XML* does not allow multiple root tags.

The first tag which contains benchmark specific information is `<benchmark>`. `hello_world` is the benchmark-name which can be used to identify the benchmark (e.g. when there are multiple benchmarks inside a single input file, or when different benchmarks use the same run directory).

The `outpath` describes the benchmark run directory (relative to the position of the input file). This directory will be managed by *JUBE* and will be automatically created if it does not exist. The directory name and position are very important, because they are the main interface to communicate with your benchmark, after it was submitted.

Using the `<comment>` you can store some benchmark related comments inside the benchmark directory. You can also use normal *XML*-comments to structure your input-file:

```
<!-- your comment -->
```

In this benchmark a `<parameterset>` is used to store the single `<parameter name="hello_str">`. The name of the parameter should contain only letters, numbers (should not be the first character) or the `_` (like a normal *Python* identifier). The name of the parameterset must be unique (relative to the current benchmark). In further examples we will see that there are more types of sets, which can be distinguished by their names. Also the name of the parameter must be unique (relative to the parameterset).

The `<step>` contains the operation tasks. The name must be unique. It can use different types of existing sets. All used sets must be given by name using the `<use>`. There can be multiple `<use>` inside the same `<step>` and also multiple names within the same `<use>` are allowed (separated by `,`). Only sets, which are explicitly used, are available inside the step! The `<do>` contains a single **shell command**. This command will run inside of a sandbox directory environment (inside the `outpath` directory tree). The step and its corresponding *parameter space* is named *workpackage*.

Available parameters can be used inside the shell commands. To use a parameter you have to write

```
$parametername
```

or

```
${parametername}
```

The brackets must be used if you want variable concatenation. `$hello_strtest` will not be replaced, `${hello_str}test` will be replaced. If a parameter does not exist or isn't available the variable will not be replaced! If you want to use `$` inside your command, you have to write `$$` to mask the symbol. Parameter substitution will be done before the normal shell substitution!

To run the benchmark just type:

```
>>> jube run hello_world.xml
```

This benchmark will produce the following output:

```
1 #####
2 # benchmark: hello_world
3 # id: 0
4 #
5 # A simple hello world
6 #####
7
8 Running workpackages (#=done, 0=wait, E=error):
9 ##### ( 1/ 1)
10
11 | stepname | all | open | wait | error | done |
12 |-----|-----|-----|-----|-----|-----|
13 | say_hello | 1 | 0 | 0 | 0 | 1 |
14
15 >>>> Benchmark information and further useful commands:
16 >>>> id: 0
17 >>>> handle: bench_run
18 >>>> dir: bench_run/000000
19 >>>> analyse: jube analyse bench_run --id 0
20 >>>> result: jube result bench_run --id 0
21 >>>> info: jube info bench_run --id 0
22 >>>> log: jube log bench_run --id 0
23 #####
```

As you can see, there was a single step `say_hello`, which runs one shell command `echo $hello_str` that will be expanded to `echo Hello World`.

The **id** is (in addition to the benchmark directory `handle`) an important number. Every benchmark run will get a new unique **id** inside the benchmark directory.

Inside the benchmark directory you will see the following structure:

```
1 bench_run          # the given outpath
2 |
3 +- 000000          # the benchmark id
4 |
5 +- configuration.xml # the stored benchmark configuration
6 +- workpackages.xml  # workpackage information
7 +- run.log           # log information
8 +- 000000_say_hello # the workpackage
9 |
10 +- done            # workpackage finished marker
```

(continues on next page)

(continued from previous page)

```

11      +- work          # user sandbox folder
12      |
13      +- stderr        # standard error messages of used shell commands
14      +- stdout        # standard output of used shell commands

```

stdout will contain Hello World in this example case.

2.5 Help

JUBE contains a command line based help functionality:

```
>>> jube help <keyword>
```

By using this command you will have direct access to all keywords inside the *glossary*.

Another useful command is the `info` command. It will show you information concerning your existing benchmarks:

```

1 # display a list of existing benchmarks
2 >>> jube info <benchmark-directory>
3 # display information about given benchmark
4 >>> jube info <benchmark-directory> -- id <id>
5 # display information about a step inside the given benchmark
6 >>> jube info <benchmark-directory> -- id <id> --step <stepname>

```

The third, also very important, functionality is the **logger**. Every `run`, `continue`, `analyse` and `result` execution will produce log information inside your benchmark directory. This file contains much useful debugging output.

You can easily access these log files by using the *JUBE* log viewer command:

```
>>> jube log [benchmark-directory] [--id id] [--command cmd]
```

e.g.:

```
>>> jube log bench_runs --command run
```

will display the `run.log` of the last benchmark found inside of `bench_runs`.

Log output can also be displayed during runtime by using the verbose output:

```
>>> jube -v run <input-file>
```

`-vv` can be used to display stdout output during runtime and `-vvv` will display the stdout output as well as the log output at the same time.

The verbosity level can also be set using the `JUBE_VERBOSE` environment variable. `JUBE_VERBOSE=1` is equivalent to `-v`, `JUBE_VERBOSE=2` to `-vv` and so on. If the verbosity level is set as a command line option, the environment variable is ignored.

Since the parsing step is done before creating the benchmark directory, there will be a `jube-parse.log` inside your current working directory, which contains the parser log information.

Errors within a `<do>` command will create a log entry and stop further execution of the corresponding parameter combination. Other parameter combinations will still be executed by default. *JUBE* can also stop automatically any further execution by using the `-e` option:

```
>>> jube run -e <input-file>
```

There is also a debugging mode integrated in *JUBE*:

```
>>> jube --debug <command> [other-args]
```

This mode avoids any *shell* execution but will generate a single log file (`jube-debug.log`) in your current working directory.

2.6 Parameter space creation

In this example we will show you an important feature of *JUBE*: The automatic *parameter space* generation.

The files used for this example can be found inside `examples/parameterspace`.

The input file `parameterspace.xml`:

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <jube>
3   <benchmark name="parameterspace" outpath="bench_run">
4     <comment>A parameterspace example</comment>
5
6     <!-- Configuration -->
7     <parameterset name="param_set">
8       <!-- Create a parameterspace out of two template parameter -->
9       <parameter name="number" type="int">1,2,4</parameter>
10      <parameter name="text" separator=";">Hello;World</parameter>
11    </parameterset>
12
13    <!-- Operation -->
14    <step name="say_hello">
15      <use>param_set</use> <!-- use existing parameterset -->
16      <do>echo "$text $number"</do> <!-- shell command -->
17    </step>
18  </benchmark>
19 </jube>
```

The input file `parameterspace.yaml`:

```
1 name: parameterspace
2 outpath: bench_run
3 comment: A parameterspace example
4
5 #Configuration
6 parameterset:
7   name: param_set
8   #Create a parameterspace out of two template parameter
9   parameter:
10     - {name: number, type: int, _: "1,2,4"} #comma separated integer must be quoted
11     - {name: text, separator: ;, _: Hello;World}
12
13 #Operation
14 step:
15   name: say_hello
16   use: param_set #use existing parameterset
17   do: echo "$text $number" #shell command
```

Whenever a parameter contains a `,` (this can be changed using the `separator` attribute) this parameter becomes a **template**. A step which **uses the parameterset** containing this parameter will run multiple times to iterate over all possible parameter combinations. In this example the step `say_hello` will run 6 times:

stepname	all	open	wait	error	done
say_hello	6	0	0	0	6

Every parameter combination will run in its own sandbox directory.

Another new keyword is the `type` attribute. The parameter type is not used inside the substitution process, but for sorting operations inside the `result` creation. The default type is `string`. Possible basic types are `string`, `int` and `float`.

2.7 Step dependencies

If you start writing a complex benchmark structure, you might want to have dependencies between different *steps*, for example between a compile and the execution step. *JUBE* can handle these dependencies and will also preserve the given *parameter space*.

The files used for this example can be found inside `examples/dependencies`.

The input file `dependencies.xml`:

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <jube>
3   <benchmark name="dependencies" outpath="bench_run">
4     <comment>A Dependency example</comment>
5
6     <!-- Configuration -->
7     <parameterset name="param_set">
8       <parameter name="number" type="int">1,2,4</parameter>
9     </parameterset>
10
11    <!-- Operations -->
12    <step name="first_step">
13      <use>param_set</use> <!-- use existing parameterset -->
14      <do>echo $number</do> <!-- shell command -->
15    </step>
16
17    <!-- Create a dependency between both steps -->
18    <step name="second_step" depend="first_step">
19      <do>cat first_step/stdout</do> <!-- shell command -->
20    </step>
21  </benchmark>
22 </jube>

```

The input file `dependencies.yaml`:

```

1 name: dependencies
2 outpath: bench_run
3 comment: A Dependency example
4
5 #Configuration
6 parameterset:
7   name: param_set
8   parameter: {name: number, type: int, _: "1,2,4"} #comma separated integers must
9             ↪ be quoted
10
11 #Operation
12 step:
13   - name: first_step

```

(continues on next page)

(continued from previous page)

```

13 use: param_set #use existing parameterset
14 do: echo $number #shell command
15 - name: second_step
16   depend: first_step #Create a dependency between both steps
17   do: cat first_step/stdout #shell command

```

In this example we create a dependency between `first_step` and `second_step`. After `first_step` is finished, the corresponding `second_step` will start. Steps can also have multiple dependencies (separated by `,` in the definition), but circular definitions will not be resolved. A dependency is a unidirectional link!

To communicate between a step and its dependency there is a link inside the work directory pointing to the corresponding dependency step work directory. In this example we use

```
cat first_step/stdout
```

to write the `stdout`-file content of the dependency step into the `stdout`-file of the current step.

Because the `first_step` uses a template parameter which creates three execution runs, there will also be three `second_step` runs each pointing to different `first_step`-directories:

stepname	all	open	wait	error	done
first_step	3	0	0	0	3
second_step	3	0	0	0	3

2.8 Loading files and substitution

Every step runs inside a unique sandbox directory. Usually, you will need to have external files inside this directory (e.g. the source files) and in some cases you want to change a parameter inside the file based on your current *parameter space*. There are two additional set-types which handle this behaviour inside of *JUBE*.

The files used for this example can be found inside `examples/files_and_sub`.

The input file `files_and_sub.xml`:

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <jube>
3   <benchmark name="files_and_sub" outpath="bench_run">
4     <comment>A file copy and substitution example</comment>
5
6     <!-- Configuration -->
7     <parameterset name="param_set">
8       <parameter name="number" type="int">1,2,4</parameter>
9       <parameter name="zahl" type="int">2,4,5</parameter>
10    </parameterset>
11
12    <!-- Files -->
13    <filesset name="files">
14      <copy>file.in</copy>
15    </filesset>
16
17    <!-- Substitute -->
18    <substituteset name="substitute">
19      <!-- Substitute files -->
20      <iofile in="file.in" out="file.out" />
21      <!-- Substitute commands -->
22      <sub source="#NUMBER#" dest="$number" />
23      <sub source="#[^NUMBER]#" dest="$zahl" mode="regex"/>

```

(continues on next page)

(continued from previous page)

```

24 </substituteset>
25
26 <!-- Operation -->
27 <step name="sub_step">
28   <use>param_set</use> <!-- use existing parameterset -->
29   <use>files</use>      <!-- use existing fileset -->
30   <use>substitute</use> <!-- use existing substituteset -->
31   <do>cat file.out</do> <!-- shell command -->
32 </step>
33 </benchmark>
34 </jube>

```

The input file files_and_sub.yaml:

```

1 name: files_and_sub
2 outpath: bench_run
3 comment: A file copy and substitution example
4
5 #Configuration
6 parameterset:
7   name: param_set
8   parameter:
9     - {name: number, type: int, _: "1,2,4"} #comma separated integers must be quoted
10    - {name: zahl, type: int, _: "2,4,5"} #comma separated integers must be quoted
11
12 #Files
13 fileset:
14   name: files
15   copy: file.in
16
17 #Substitute
18 substituteset:
19   name: substitute
20   iofile: {in: file.in, out: file.out}
21   sub:
22     - {source: "#NUMBER#", dest: $number} #"#" must be quoted
23     - {source: "#[^NUMBER]+#", dest: $zahl, mode: "regex"} #"#" must be quoted
24
25 #Operation
26 step:
27   name: sub_step
28   use:
29     - param_set #use existing parameterset
30     - files #use existing fileset
31     - substitute #use existing substituteset
32   do: cat file.out #shell command

```

The content of file file.in:

```

Number: #NUMBER#
Zahl: #Zahl#

```

Inside the <fileset> the current location (relative to the current input file; also absolute paths are allowed) of files is defined. <copy> specifies that the file should be copied to the sandbox directory when the fileset is used. Also a <link> option is available to create a symbolic link to the given file inside the sandbox directory.

If there are additional operations needed to *prepare* your files (e.g. expand a tar-file). You can use the <prepare>-tag inside your <fileset>.

The `<substituteset>` describes the substitution process. The `<iofile>` contains the input and output filename. The path is relative to the sandbox directory. Because we do/should not know that location we use the fileset to copy `file.in` to this directory.

The `<sub>` specifies the substitution. All occurrences of `source` will be substituted by `dest`. As you can see, you can use parameters inside the substitution. In addition to the standard text substitution (see `<sub source="#NUMBER#" ... />`), the mode attribute (introduced in JUBE version 2.6.0) also allows regular expressions to be used for the substitution (see `<sub mode="regex" source="#[^NUMBER]++#" ... />`). The regular expression in this example searches for a capitalised text enclosed in `#`. It matches the following text in the file `file.in`: `#ZAHL#`.

There is no `<use>` inside any set. The combination of all sets will be done inside the `<step>`. So if you use a parameter inside a `<sub>` you must also add the corresponding `<parameterset>` inside the `<step>` where you use the `<substituteset>`!

In the `sub_step` we use all available sets. The use order is not relevant. The normal execution process will be:

1. Parameter space expansion
2. Copy/link files
3. Prepare operations
4. File substitution
5. Run shell operations

The resulting directory-tree will be:

```
1 bench_run          # the given outpath
2 |
3 +- 000000          # the benchmark id
4 |
5 +- configuration.xml # the stored benchmark configuration
6 +- workpackages.xml # workpackage information
7 +- 000000_sub_step # the workpackage ($number = 1, $zahl = 2)
8 |
9 +- done            # workpackage finished marker
10 +- work            # user sandbox folder
11 |
12 +- stderr          # standard error messages of used shell commands
13 +- stdout          # standard output of used shell commands (Number: 1 Zahl: 2)
14 +- file.in         # the file copy
15 +- file.out        # the substituted file
16 +- 000001_sub_step # the workpackage ($number = 1, $zahl = 4)
17 |
18 +- ...
19 +- ...
```

And the content of file `file.out` in `000000_sub_step/work`:

```
Number: 1
Zahl: 2
```

2.9 Creating a result table

Finally, after running the benchmark, you will get several directories. *JUBE* allows you to parse your result files distributed over these directories to extract relevant data (e.g. walltime information) and create a result table.

The files used for this example can be found inside `examples/result_creation`.

The input file `result_creation.xml`:


```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <jube>
3   <benchmark name="result_creation" outpath="bench_run">
4     <comment>A result creation example</comment>
5
6     <!-- Configuration -->
7     <parameterset name="param_set">
8       <!-- Create a parameterspace with one template parameter -->
9       <parameter name="number" type="int">1,2,4</parameter>
10    </parameterset>
11
12    <!-- Regex pattern -->
13    <patternset name="pattern_all">
14      <pattern name="number_pat" type="int">.*? $jube_pat_int</pattern>
15    </patternset>
16    <patternset name="pattern_en">
17      <pattern name="number_pat_en" type="int">Number: $jube_pat_int</pattern>
18    </patternset>
19    <patternset name="pattern_de">
20      <pattern name="number_pat_de" type="int">Zahl: $jube_pat_int</pattern>
21    </patternset>
22
23    <!-- Operation -->
24    <step name="write_number">
25      <use>param_set</use> <!-- use existing parameterset -->
26      <do>echo "Number: $number" > en </do> <!-- shell command -->
27      <do>echo "Zahl: $number" > de </do> <!-- shell command -->
28    </step>
29
30    <!-- Analyse -->
31    <analyser name="analyse">
32      <use>pattern_all</use> <!-- use this patternset for all files -->
33      <analyse step="write_number">
34        <file use="pattern_en">en</file> <!-- file en will be scanned with pattern_
35    ↪ all and pattern_en -->
36        <file use="pattern_de">de</file> <!-- file de will be scanned with pattern_
37    ↪ all and pattern_de -->
38      </analyse>
39    </analyser>
40
41    <!-- Create result table -->
42    <result>
43      <use>analyse</use> <!-- use existing analyser -->
44      <table name="result" style="pretty" sort="number">
45        <column>number</column>
46        <column>number_pat</column> <!-- Column with title same as pattern name -->
47        <column title="Number">number_pat_en</column> <!-- Column with costum title --
48    ↪
49        <column title="Zahl">number_pat_de</column>
50      </table>
51    </result>
52  </benchmark>
53 </jube>

```

The input file result_creation.yaml:

```
1 name: result_creation
```

(continues on next page)

(continued from previous page)

```

2 outpath: bench_run
3 comment: A result creation example
4
5 #Configuration
6 parameterset:
7   name: param_set
8   #Create a parameterspace with one template parameter
9   parameter: {name: number, type: int, _: "1,2,4"} # comma separated integer must be_
10  ↪quoted
11
12 #Regex pattern
13 patternset:
14 - name: pattern_all
15   pattern:
16   - {name: number_pat, type: int, _: ".*? $jube_pat_int"} # "?" must be quoted
17 - name: pattern_en
18   pattern:
19   - {name: number_pat_en, type: int, _: "Number: $jube_pat_int"} # ":" must be quoted
20 - name: pattern_de
21   pattern:
22   - {name: number_pat_de, type: int, _: "Zahl: $jube_pat_int"}
23
24 #Operation
25 step:
26   name: write_number
27   use: param_set #use existing parameterset
28   do:
29   - 'echo "Number: $number" > en'
30   - 'echo "Zahl: $number" > de' #shell commands
31
32 #Analyse
33 analyser:
34 - name: analyse
35   use: pattern_all #use existing patternset for all files
36   analyse:
37     step: write_number
38     file:
39     - use: pattern_en #use patternset only for this file
40     - use: pattern_de
41     - use: pattern_de
42
43 #Create result table
44 result:
45   use: analyse #use existing analyser
46   table:
47     name: result
48     style: pretty
49     sort: number
50     column:
51     - number
52     - number_pat # Column with title same as pattern name
53     - { title: "Number", _: number_pat_en } # Column with costum title
54     - { title: "Zahl", _: number_pat_de }

```

Using <parameterset> and <step> we create three *workpackages*. Each writing Number: \$number to en and Zahl: \$number to de.

Now we want to parse these `en` and `de` files to extract information (in this example case the written number). First of all we have to declare a `<patternset>`. Here we can describe a set of `<pattern>`. A `<pattern>` is a regular expression which will be used to parse your result files and search for a given string. In this example we have the `<pattern>` `number_pat` which matches both file contents and the more specific version `number_pat_en` and `number_pat_de`. The name of the pattern must be unique (based on the usage of the `<patternset>`). The `type` is optional. It is used when the extracted data will be sorted. The regular expression can contain other patterns or parameters. The example uses `$jube_pat_int` which is a *JUBE default pattern* matching integer values. The pattern can contain a group, given by brackets (`...`), to declare the extraction part (`$jube_pat_int` already contains these brackets).

E.g. `$jube_pat_int` and `$jube_pat_fp` are defined in the following way:

```
<pattern name="jube_pat_int" type="int">([+-]?\d+)</pattern>
<pattern name="jube_pat_fp" type="float">([+-]?\d*\.\.?d+(?:[eE][+-]?\d+)?</pattern>
```

A complete list of predefined patterns is in the glossary. If there are multiple matches inside a single file you can add a *reduce option*. By default, only the first match will be extracted.

To use your `<patternset>` you have to specify the files which should be parsed. This can be done using the `<analyser>`. It uses relevant `patternsets`. Inside the `<analyse>` a step-name and a file inside this step is given. Every workpackage file combination and every analyser will create its own result entry. Additional `<patternset>` can be used inside the `<file>` tag in order to apply this set only to one file.

The analyser automatically knows all parameters which were used in the given step and in depending steps. There is no `<use>` option to include additional `<parameterset>` that have not been already used within the analysed `<step>`.

To run the analyse you have to write:

```
>>> jube analyse bench_run
```

The analyse data will be stored inside the benchmark directory.

The last part is the result table creation. Here you have to use an existing analyser. The `<column>` contains a pattern or a parameter name. `sort` is the optional sorting order (separated by `,`). The `style` attribute can be `csv`, `pretty` or `aligned` to get different ASCII representations.

To create the result table you have to write:

```
>>> jube result bench_run -i last
```

If you run the `result` command for the first time, the `analyse` step will be executed automatically, if it wasn't executed before. So it is not necessary to run the separate `analyse` step all the time. However you need the separate `analyse` if you want to force a re-run of the `analyse` step, otherwise only the stored values of the previous `analyse` will be used in the `result` step.

The `analyse` and `result` instructions can be combined within one single command:

```
>>> jube result bench_run -a
```

The result table will be written to `STDOUT` and into a `result.dat` file inside `bench_run/<id>/result`.

The `--id` or `-i` option can be used to output the result of a specific benchmark ID. If this option is not set, the result of the last benchmark will be shown (i.e. the default is `last`).

Output of the given example:

number	number_pat	Number	Zahl
1	1	1	1
2	2	2	2
4	4	4	4

If you want to hide or show only certain output columns, you can use the `--select` and `--exclude` options since JUBE version 2.6.0. These options take parameter and pattern names as arguments. For example, you can use the following commands to display only the `number` column of the result table:

```
>>> jube result bench_run --select number
```

or

```
>>> jube result bench_run --exclude number_pat number_pat_en number_pat_de
```

The specified columns are hidden not only in the output, but also in the results file. Both options can be given on the command line, and only the columns included in `--select` and not `--exclude` will be displayed.

This was the last example of the basic *JUBE* tutorial. Next you can start the [advanced tutorial](#) to get more information about including external sets, jobssystem representation and scripting parameter.

ADVANCED TUTORIAL

This tutorial demonstrates more detailed functions and tools of *JUBE*. If you want a basic overview you should read the general *JUBE tutorial* first.

3.1 Schema validation

To validate your *YAML* based input files you can use schema validation. You can find `jube.json` in the `contrib/schema` folder.

To validate your *XML* based input files you can use DTD or schema validation. You can find `jube.dtd`, `jube.xsd` and `jube.rnc` in the `contrib/schema` folder. You have to add these schema information to your input files which you want to validate.

DTD usage:

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <!DOCTYPE jube SYSTEM "<jube.dtd path>">
3 <jube>
4 ...
```

Schema usage:

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <jube xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
3   xsi:noNamespaceSchemaLocation="<jube.xsd path>">
4 ...
```

RELAX NG Compact Syntax (RNC for emacs nxml-mode) usage:

In order to use the provided rnc schema file `schema/jube.rnc` in emacs open an xml file and use C-c C-s C-f or M-x `rng-set-schema-file-and-validate` to choose the rnc file. You can also use M-x `customize-variable rng-schema-locating-files` after you loaded nxml-mode to customize the default search paths to include `jube.rnc`. After successful parsing emacs offers to automatically create a `schema.xml` file which looks like

```
1 <?xml version="1.0"?>
2 <locatingRules xmlns="http://thaiopensource.com/ns/locating-rules/1.0">
3   <uri resource="jube-file.xml" uri="../schema/jube.rnc"/>
4 </locatingRules>
```

The next time you open the same xml file emacs will find the correct rnc for the validation based on `schema.xml`.

Example validation tools:

- eclipse (using DTD or schema)
- emacs (using RELAX NG)
- xmllint:

- For validation (using the DTD):

```
>>> xmllint --noout --valid <xml input file>
```

- For validation (using the DTD and Schema):

```
>>> xmllint --noout --valid --schema <schema file> <xml input file>
```

3.2 Scripting parameter

In some cases it is needed to create a parameter which is based on the value of another parameter. In this case you can use a scripting parameter.

The files used for this example can be found inside `examples/scripting_parameter`.

The input file `scripting_parameter.xml`:

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <jube>
3   <benchmark name="scripting_parameter" output="bench_run">
4     <comment>A scripting parameter example</comment>
5
6     <!-- Configuration -->
7     <parameterset name="param_set">
8       <!-- Normal template -->
9       <parameter name="number" type="int">1,2,4</parameter>
10      <!-- A template created by a scripting parameter-->
11      <parameter name="additional_number" mode="python" type="int">
12        ", ".join(str(a*${number}) for a in [1,2])
13      </parameter>
14      <!-- A scripting parameter -->
15      <parameter name="number_mult" mode="python" type="float">
16        ${number}*${additional_number}
17      </parameter>
18      <!-- Reuse another parameter -->
19      <parameter name="text">Number: $number</parameter>
20    </parameterset>
21
22    <!-- Operation -->
23    <step name="operation">
24      <use>param_set</use> <!-- use existing parameterset -->
25      <!-- shell commands -->
26      <do>echo "number: $number, additional_number: $additional_number"</do>
27      <do>echo "number_mult: $number_mult, text: $text"</do>
28    </step>
29  </benchmark>
30 </jube>
```

The input file `scripting_parameter.yaml`:

```
1 name: scripting_parameter
2 output: bench_run
3 comment: A scripting parameter example
4
5 #Configuration
6 parameterset:
7   name: param_set
8   parameter:
```

(continues on next page)

(continued from previous page)

```

9      #Normal template
10     - {name: number, type: int, _: "1,2,4"}
11     #A template created by a scripting parameter
12     - {name: additional_number, mode: python, type: int, _: '"'.join(str(a*${number}
13     ↪) for a in [1,2])' }
14     #A scripting parameter
15     - {name: number_mult, mode: python, type: float, _: "${number}*${additional_
16     ↪number}"}
17     #Reuse another parameter
18     - {name: text, _: "Number: $number"}
19
20 #Operation
21 step:
22   name: operation
23   use: param_set #use existing parameterset
24   do:
25     - 'echo "number: $number, additional_number: $additional_number"'
26     - 'echo "number_mult: $number_mult, text: $text"'

```

In this example we see four different parameters.

- `number` is a normal template which will be expanded to three different *workpackages*.
- `additional_number` is a scripting parameter which creates a new template and bases on `number`. The mode is set to the scripting language (python, perl and shell are allowed). The additional type is optional and declares the result type after evaluating the expression. The type is only used by the sort algorithm in the result step. It is not possible to create a template of different scripting parameters. Because of this second template we will get six different *workpackages*.
- `number_mult` is a small calculation. You can use any other existing parameters (which are used inside the same step).
- `text` is a normal parameter which uses the content of another parameter. For a simple concatenation parameter you do not need a scripting parameter.

For this example we will find the following output inside the `run.log`-file:

```

1  ===== operation =====
2  >>> echo "number: 1, additional_number: 1"
3  >>> echo "number_mult: 1, text: Number: 1"
4  ===== operation =====
5  >>> echo "number: 1, additional_number: 2"
6  >>> echo "number_mult: 2, text: Number: 1"
7  ===== operation =====
8  >>> echo "number: 2, additional_number: 2"
9  >>> echo "number_mult: 4, text: Number: 2"
10 ===== operation =====
11 >>> echo "number: 2, additional_number: 4"
12 >>> echo "number_mult: 8, text: Number: 2"
13 ===== operation =====
14 >>> echo "number: 4, additional_number: 4"
15 >>> echo "number_mult: 16, text: Number: 4"
16 ===== operation =====
17 >>> echo "number: 4, additional_number: 8"
18 >>> echo "number_mult: 32, text: Number: 4"

```

Implicit Perl or Python scripting inside the `<do>` or any other position is not possible. If you want to use some scripting expressions you have to create a new parameter.

3.3 Scripting pattern

Similar to the *Scripting parameter*, also different patterns, or patterns and parameters can be combined. For this a scripting pattern can be created by using the `mode=` attribute in the same way as it is used for the *Scripting parameter*.

All scripting patterns are evaluated at the end of the analyse part. Each scripting pattern is evaluated once. If there are multiple matches as described in the *Statistic pattern values* section, only the resulting statistical pattern is available (not each individual value). Scripting pattern do not create statistic values by themselves.

In addition the `default=` attribute can be used to set a default pattern value, if the value can't be found during the analysis.

The files used for this example can be found inside `examples/scripting_pattern`.

The input file `scripting_pattern.xml`:

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <jube>
3   <benchmark name="scripting_pattern" outpath="bench_run">
4     <comment>A scripting_pattern example</comment>
5
6     <!-- Configuration -->
7     <parameterset name="param_set">
8       <parameter name="value" type="int">0,1,2</parameter>
9     </parameterset>
10
11    <!-- Operation -->
12    <step name="operation">
13      <use>param_set</use>
14      <do>echo "$value"</do>
15    </step>
16
17    <!-- Pattern to extract -->
18    <patternset name="pattern_set">
19      <!-- A normal pattern -->
20      <pattern name="value_pat" type="int">$jube_pat_int</pattern>
21      <!-- A combination of a pattern and a parameter -->
22      <pattern name="dep_pat" type="int" mode="python">$value_pat+$value</pattern>
23      <!-- This pattern is not available -->
24      <pattern name="missing_pat" type="int">
25        pattern_not_available: $jube_pat_int
26      </pattern>
27      <!-- The combination will fail (create NaN) -->
28      <pattern name="missing_dep_pat" type="int" mode="python">
29        $missing_pat*$value
30      </pattern>
31      <!-- Default value for missing pattern -->
32      <pattern name="missing_pat_def" type="int" default="0">
33        pattern_not_available: $jube_pat_int
34      </pattern>
35      <!-- Combination of default value and parameter -->
36      <pattern name="missing_def_dep_pat" type="int" mode="python">
37        $missing_pat_def*$value
38      </pattern>
39    </patternset>
40
41    <analyser name="analyse">
42      <use>pattern_set</use>

```

(continues on next page)

(continued from previous page)

```

43     <analyse step="operation">
44         <file>stdout</file>
45     </analyse>
46 </analyser>
47
48 <!-- result table creation -->
49 <result>
50     <use>analyse</use>
51     <table name="result" style="pretty">
52         <column>value</column>
53         <column>value_pat</column>
54         <column>dep_pat</column>
55         <column>missing_pat</column>
56         <column>missing_dep_pat</column>
57         <column>missing_pat_def</column>
58         <column>missing_def_dep_pat</column>
59     </table>
60 </result>
61 </benchmark>
62 </jube>

```

The input file scripting_pattern.yaml:

```

1  name: scripting_pattern
2  outpath: bench_run
3  comment: A scripting_pattern example
4
5  #Configuration
6  parameterset:
7      name: param_set
8      parameter: {name: value, type: int, _: "0,1,2"}
9
10 #Operation
11 step:
12     name: operation
13     use: param_set
14     do: echo "$value"
15
16 #Pattern to extract
17 patternset:
18     name: pattern_set
19     pattern:
20         #A normal pattern
21         - {name: value_pat, type: int, _: $jube_pat_int}
22         #A combination of a pattern and a parameter
23         - {name: dep_pat, type: int, mode: python, _: $value_pat+$value}
24         #This pattern is not available
25         - {name: missing_pat, type: int, _: "pattern_not_available: $jube_pat_int"}
26         #The combination will fail (create NaN)
27         - {name: missing_dep_pat, type: int, mode: python, _: $missing_pat*$value}
28         #Default value for missing pattern
29         - {name: missing_pat_def, type: int, default: 0, _: "pattern_not_available: $jube_
30         ↪pat_int"}
31         #Combination of default value and parameter
32         - {name: missing_def_dep_pat, type: int, mode: python, _: $missing_pat_def*$value}

```

(continues on next page)

(continued from previous page)

```

33 analyser:
34   name: analyse
35   use: pattern_set
36   analyse:
37     step: operation
38     file: stdout
39
40 #result table creation
41 result:
42   use: analyse
43   table:
44     name: result
45     style: pretty
46     column: [value, value_pat, dep_pat, missing_pat, missing_dep_pat, missing_pat_def,
    ↪ missing_def_dep_pat]

```

It will create the following output:

```

1 | value | value_pat | dep_pat | missing_pat | missing_dep_pat | missing_pat_def |
  ↪ missing_def_dep_pat |
2 |-----|-----|-----|-----|-----|-----|-----|
  ↪-----|
3 |    0 |    0 |    0 |    |    nan |    0 |
  ↪    0 |
4 |    1 |    1 |    2 |    |    nan |    0 |
  ↪    0 |
5 |    2 |    2 |    4 |    |    nan |    0 |
  ↪    0 |

```

3.4 Statistic pattern values

Normally a pattern should only match a single entry in your result files. But sometimes there are multiple similar entries (e.g. if the benchmark uses some iteration feature).

JUBE will create the statistical values `first`, `last`, `min`, `max`, `avg`, `std`, `cnt` and `sum` automatically. To use these values, the user have to specify the pattern name followed by `_<statistic_option>`, e.g. `pattern_name_last` (the `pattern_name` itself will always be the first match).

An example for multiple matches and the statistic values can be found in `examples/statistic`.

The input file `statistic.xml`:

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <jube>
3   <benchmark name="reduce_example" outpath="bench_run">
4     <comment>A result reduce example</comment>
5
6     <!-- Regex pattern -->
7     <patternset name="pattern">
8       <pattern name="number_pat" type="int">$jube_pat_int</pattern>
9     </patternset>
10
11    <!-- Operation -->
12    <step name="write_some_numbers">
13      <do>echo "1 2 3 4 5 6 7 8 9 10"</do> <!-- shell command -->
14    </step>
15

```

(continues on next page)

(continued from previous page)

```

16 <!-- Analyse -->
17 <analyser name="analyse">
18   <use>pattern</use> <!-- use existing patternset -->
19   <analyse step="write_some_numbers">
20     <file>stdout</file> <!-- file which should be scanned -->
21   </analyse>
22 </analyser>
23
24 <!-- Create result table -->
25 <result>
26   <use>analyse</use> <!-- use existing analyser -->
27   <table name="result" style="pretty">
28     <column>number_pat</column> <!-- first match -->
29     <column>number_pat_first</column> <!-- first match -->
30     <column>number_pat_last</column> <!-- last match -->
31     <column>number_pat_min</column> <!-- min of all matches -->
32     <column>number_pat_max</column> <!-- max of all matches -->
33     <column>number_pat_sum</column> <!-- sum of all matches -->
34     <column>number_pat_cnt</column> <!-- number of matches -->
35     <column>number_pat_avg</column> <!-- avg of all matches -->
36     <column format=".2f">number_pat_std</column> <!-- std of all matches -->
37   </table>
38 </result>
39 </benchmark>
40 </jube>

```

The input file statistic.yaml:

```

1 name: reduce_example
2 outpath: bench_run
3 comment: A result reduce example
4
5 #Regex pattern
6 patternset:
7   name: pattern
8   pattern: {name: number_pat, type: int, _: $jube_pat_int}
9
10 #Operation
11 step:
12   name: write_some_numbers
13   do: echo "1 2 3 4 5 6 7 8 9 10" #shell command
14
15 #Analyse
16 analyser:
17   name: analyse
18   use: pattern #use existing patternset
19   analyse:
20     step: write_some_numbers
21     file: stdout #file which should be scanned
22
23 #Create result table
24 result:
25   use: analyse #use existing analyser
26   table:
27     name: result
28     style: pretty

```

(continues on next page)

(continued from previous page)

```

29 column:
30   - number_pat #first match
31   - number_pat_first #first match
32   - number_pat_last #last match
33   - number_pat_min #min of all matches
34   - number_pat_max #max of all matches
35   - number_pat_sum #sum of all matches
36   - number_pat_cnt #number of matches
37   - number_pat_avg #avg of all matches
38   - {_: number_pat_std, format: .2f} #std of all matches

```

It will create the following output:

```

| number_pat | number_pat_last | number_pat_min | number_pat_max | number_pat_sum |
↪ number_pat_cnt | number_pat_avg | number_pat_std |
|-----|-----|-----|-----|-----|
↪-----|-----|-----|
|          1 |          10 |          1 |          10 |          55 |
↪          10 |          5.5 |          3.03 |

```

3.5 Jobsystem

In most cases you want to submit jobs by *JUBE* to your local jobsystem. You can use the normal file access and substitution system to prepare your jobfile and send it to the jobsystem. *JUBE* also provide some additional features.

The files used for this example can be found inside `examples/jobsystem`.

The input jobsystem file `job.run.in` for *Torque/Moab* (you can easily adapt your personal jobscript):

```

1  #!/bin/bash -x
2  #MSUB -l nodes=#NODES#:ppn=#PROCS_PER_NODE#
3  #MSUB -l walltime=#WALLTIME#
4  #MSUB -e #ERROR_FILEPATH#
5  #MSUB -o #OUT_FILEPATH#
6  #MSUB -M #MAIL_ADDRESS#
7  #MSUB -m #MAIL_MODE#
8
9  ### start of jobscript
10
11 #EXEC#
12 touch #READY#

```

The *JUBE* input file `jobsystem.xml`:

```

1  <?xml version="1.0" encoding="UTF-8"?>
2  <jube>
3    <benchmark name="jobsystem" outpath="bench_run">
4      <comment>A jobsystem example</comment>
5
6      <!-- benchmark configuration -->
7      <parameterset name="param_set">
8        <parameter name="number" type="int">1,2,4</parameter>
9      </parameterset>
10
11     <!-- Job configuration -->
12     <parameterset name="executeset">

```

(continues on next page)

(continued from previous page)

```

13     <parameter name="submit_cmd">msub</parameter>
14     <parameter name="job_file">job.run</parameter>
15     <parameter name="nodes" type="int">1</parameter>
16     <parameter name="walltime">00:01:00</parameter>
17     <parameter name="ppn" type="int">4</parameter>
18     <parameter name="ready_file">ready</parameter>
19     <parameter name="mail_mode">abe</parameter>
20     <parameter name="mail_address"></parameter>
21     <parameter name="err_file">stderr</parameter>
22     <parameter name="out_file">stdout</parameter>
23     <parameter name="exec">echo $number</parameter>
24 </parameterset>
25
26 <!-- Load jobfile -->
27 <fileset name="files">
28     <copy>${job_file}.in</copy>
29 </fileset>
30
31 <!-- Substitute jobfile -->
32 <substituteset name="sub_job">
33     <iofile in="${job_file}.in" out="$job_file" />
34     <sub source="#NODES#" dest="$nodes" />
35     <sub source="#PROCS_PER_NODE#" dest="$ppn" />
36     <sub source="#WALLTIME#" dest="$walltime" />
37     <sub source="#ERROR_FILEPATH#" dest="$err_file" />
38     <sub source="#OUT_FILEPATH#" dest="$out_file" />
39     <sub source="#MAIL_ADDRESS#" dest="$mail_address" />
40     <sub source="#MAIL_MODE#" dest="$mail_mode" />
41     <sub source="#EXEC#" dest="$exec" />
42     <sub source="#READY#" dest="$ready_file" />
43 </substituteset>
44
45 <!-- Operation -->
46 <step name="submit" work_dir="$$SCRATCH/jobsystem_bench_${jube_benchmark_id}_${jube_wp_id}" >
47     <use>param_set</use>
48     <use>executeset</use>
49     <use>files,sub_job</use>
50     <do done_file="$ready_file">$submit_cmd $job_file</do> <!-- shell command -->
51 </step>
52 </benchmark>
53 </jube>

```

The JUBE input file jobsystem.yaml:

```

1 name: jobsystem
2 outpath: bench_run
3 comment: A jobsystem example
4
5 parameterset:
6     #benchmark configuration
7     - name: param_set
8       parameter: {name: number, type: int, _: "1,2,4"} #comma separated integer must
9     #Job configuration
10    - name: executeset

```

(continues on next page)

(continued from previous page)

```

11 parameter:
12     - {name: submit_cmd, "_": msub}
13     - {name: job_file, "_": job.run}
14     - {name: nodes, type: int, "_": 1}
15     - {name: walltime, "_": "00:01:00"} #: must be quoted
16     - {name: ppn, type: int, "_": 4}
17     - {name: ready_file, "_": ready}
18     - {name: mail_mode, "_": abe}
19     - {name: mail_address}
20     - {name: err_file, "_": stderr}
21     - {name: out_file, "_": stdout}
22     - {name: exec, "_": echo $number}
23
24 #Load jobfile
25 fileset:
26     name: files
27     copy: ${job_file}.in
28
29 substituteset:
30     name: sub_job
31     iofile: {in: "${job_file}.in", out: $job_file} #attributes with {} must be quoted
32     sub:
33         - {source: "#NODES#", dest: $nodes}
34         - {source: "#PROCS_PER_NODE#", dest: $ppn}
35         - {source: "#WALLTIME#", dest: $walltime}
36         - {source: "#ERROR_FILEPATH#", dest: $err_file}
37         - {source: "#OUT_FILEPATH#", dest: $out_file}
38         - {source: "#MAIL_ADDRESS#", dest: $mail_address}
39         - {source: "#MAIL_MODE#", dest: $mail_mode}
40         - {source: "#EXEC#", dest: $exec}
41         - {source: "#READY#", _ : $ready_file } # _ can be used here as well instead of ↵
42         ↵dest (should be used for multiline output)
43
44 #Operation
45 step:
46     name: submit
47     work_dir: "$$WORK/jobsystem_bench_${jube_benchmark_id}_${jube_wp_id}"
48     use: [param_set,executeset,files,sub_job]
49     do:
50         done_file: $ready_file
51         _ : $submit_cmd $job_file #shell command

```

As you can see the jobfile is very general and several parameters will be used for replacement. By using a general jobfile and the substitution mechanism you can control your jobsystem directly out of your *JUBE* input file. \$\$ is used for *Shell* substitutions instead of *JUBE* substitution (see [Environment handling](#)).

The submit command is a normal *Shell* command so there are no special *JUBE* tags to submit a job.

There are two new attributes:

- `done_file` inside the `<do>` allows you to set a filename/path to a file which should be used by the jobfile to mark the end of execution. *JUBE* does not know when the job ends. Normally it will return when the *Shell* command was finished. When using a jobsystem the user usually have to wait until the jobfile is executed. If *JUBE* found a `<do>` containing a `done_file` attribute *JUBE* will return directly and will not continue automatically until the `done_file` exists. If you want to check the current status of your running steps and continue the benchmark process if possible you can type:

```
>>> jube continue bench_run
```

This will continue your benchmark execution (`bench_run` is the benchmarks directory in this example). The position of the `done_file` is relatively seen towards the work directory.

- `work_dir` can be used to change the sandbox work directory of a step. In normal cases *JUBE* checks that every work directory gets a unique name. When changing the directory the user must select a unique name by his own. For example he can use `$jube_benchmark_id` and `$jube_wp_id`, which are *JUBE internal parameters* and will be expanded to the current benchmark and workpackage ids. Files and directories out of a given `<fileset>` will be copied into the new work directory. Other automatic links, like the dependency links, will not be created!

You will see this Output after running the benchmark:

stepname	all	open	wait	error	done
submit	3	0	3	0	0

and this output after running the `continue` command (after the jobs where executed):

stepname	all	open	wait	error	done
submit	3	0	0	0	3

You have to run `continue` multiple times if not all `done_file` were written when running `continue` for the first time.

3.6 Include external data

As you have seen in the example before a benchmark can become very long. To structure your benchmark you can use multiple files and reuse existing sets. There are three different include features available.

The files used for this example can be found inside `examples/include`.

The include file `include_data.xml`:

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <jube>
3   <parameterset name="param_set">
4     <parameter name="number" type="int">1,2,4</parameter>
5   </parameterset>
6
7   <parameterset name="param_set2">
8     <parameter name="text">Hello</parameter>
9   </parameterset>
10
11   <dos>
12     <do>echo Test</do>
13     <do>echo $number</do>
14   </dos>
15 </jube>
```

The include file `include_data.yaml`:

```

1 parameterset:
2   - name: param_set
3     parameter: {name: number, type: int, _: "1,2,4"}
4   - name: param_set2
5     parameter: {name: text, _: Hello}
```

(continues on next page)

(continued from previous page)

```

6
7 dos:
8   - echo Test
9   - echo $number

```

All files which contain data to be included must use the *XML*-format. The include files can have a user specific structure (there can be no valid *JUBE* tags like `<dos>`), but the structure must be allowed by the searching mechanism (see below). The resulting file must have a valid *JUBE* structure.

The main file `main.xml`:

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <jube>
3   <benchmark name="include" outpath="bench_run">
4     <comment>A include example</comment>
5
6     <!-- use parameterset out of an external file and add a additional parameter -->
7     <parameterset name="param_set" init_with="include_data.xml">
8       <parameter name="foo">bar</parameter>
9     </parameterset>
10
11     <!-- Operation -->
12     <step name="say_hello">
13       <use>param_set</use> <!-- use existing parameterset -->
14       <use from="include_data.xml">param_set2</use> <!-- out of an external file -->
15       <do>echo $foo</do> <!-- shell command -->
16       <include from="include_data.xml" path="dos/do" /> <!-- include all available_
17     </step>
18   </benchmark>
19 </jube>

```

The main file `main.yaml`:

```

1 name: include
2 outpath: bench_run
3 comment: A include example
4
5 #use parameterset out of an external file and add a additional parameter
6 parameterset:
7   name: param_set
8   init_with: include_data.yaml
9   parameter: {name: foo, _: bar}
10
11 #Operation
12 step:
13   name: say_hello
14   use:
15     - param_set #use existing parameterset
16     - from: include_data.yaml
17       _: param_set2 #out of an external file
18   do:
19     - echo $foo
20     - !include include_data.yaml:["dos"] #include all available tag

```

In these file there are three different include types:

The `init_with` can be used inside any set definition. Inside the given file the search mechanism will search for

the same set (same type, same name), will parse its structure (this must be *JUBE* valid) and copy the content to `main.xml`. Inside `main.xml` you can add additional values or overwrite existing ones. If your include-set uses a different name inside your include file you can use `init_with="filename.xml:old_name"`. It is possible to mix *YAML* based include files with *XML* files and vice versa.

The second method is the `<use from="...">`. This is mostly the same like the `init_with` structure, but in this case you are not able to add or overwrite some values. The external set will be used directly. There is no set-type inside the `<use>`, because of that, the set's name must be unique inside the include-file. The remote file can use the *YAML* or the *XML* format.

The last method is the most generic include. The include mechanic is the only element in *JUBE* which works slightly different in *YAML* and *XML* based files.

In *XML* based files by using `<include />` you can copy any *XML*-nodes you want to your main-*XML* file. The included file can provide tags which are not *JUBE*-conform but it must be a valid *XML*-file (e.g. only one root node allowed). The resulting main configuration file must be completely *JUBE* valid. The path is optional and can be used to select a specific node set (otherwise the root-node itself will be included). The `<include />` is the only include-method that can be used to include any tag you want. The `<include />` will copy all parts without any changes. The other include types will update path names, which were relative to the include-file position.

In *YAML* based files the prefix `! include` is used followed by the file name. The file must be a *YAML* file, which will be opened and parsed. The second block `:[ "dos" ]` can be used to select any subset of data of the full dictionary, any Python syntax is allowed for this selection. Finally it is possible to also specify a third block which allows full Python list comprehensions. `_` is the match of the selection before, e.g.: `!include include_data.yaml:[ "dos" ]:[i for i in _ if "Test" in i]`. In contrast to the *XML* based include it isn't possible to mix lists or dictionaries out of different files, each key can only handle a single include.

To run the benchmark you can use the normal command:

```
>>> jube run main.xml
```

It will search for the files to include inside four different positions, in the following order:

- inside a directory given over the command line interface:

```
>>> jube run --include-path some_path another_path -- main.xml
```

- inside any path given by an `<include-path>`- or `include-path:-` tag in *XML* or *YAML*, respectively:

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <jube>
3   <include-path>
4     <path>some_path</path>
5     <path>another_path</path>
6   </include-path>
7   ...
8 </jube>
```

```
1 ...
2 include-path:
3   path:
4     - "some path"
5     - "another path"
6   ...
```

- inside any path given with the `JUBE_INCLUDE_PATH` environment variable (see [Configuration](#)):

```
>>> export JUBE_INCLUDE_PATH=some_path:another_path
```

- inside the same directory of your `main.xml`

JUBE stops searching as soon as it finds the file to include, or gives an error if the file is not found.

3.7 Tagging

Tagging is an easy way to hide selected parts of your input file.

The files used for this example can be found inside `examples/tagging`.

The input file `tagging.xml`:

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <jube>
3   <tags forced="true">
4     <check_tags>deu|eng</check_tags>
5     <tag name="deu">For german strings</tag>
6     <tag name="eng">For english strings</tag>
7   </tags>
8
9   <benchmark name="tagging" outpath="bench_run">
10     <comment>Tags as logical combination</comment>
11
12     <!-- Configuration -->
13     <parameterset name="param_set">
14       <parameter name="hello_str" tag="!deu+eng">Hello</parameter>
15       <parameter name="hello_str" tag="deu|!eng">Hallo</parameter>
16       <parameter name="world_str" tag="eng">World</parameter>
17     </parameterset>
18
19     <!-- Operation -->
20     <step name="say_hello">
21       <use>param_set</use> <!-- use existing parameterset -->
22       <do>echo '$hello_str $world_str'</do> <!-- shell command -->
23     </step>
24   </benchmark>
25 </jube>

```

The input file `tagging.yaml`:

```

1 tags:
2   check_tags: deu|eng #check if tag deu or eng was set
3   forced: True
4   tag:
5     - {name: deu, _: For german strings}
6     - {name: eng, _: For english strings}
7
8 name: tagging
9 outpath: bench_run
10 comment: Tags as logical combination
11
12 #Configuration
13 parameterset:
14   name: param_set
15   parameter:
16     - {name: hello_str, tag: "!deu+eng", _: Hello}
17     - {name: hello_str, tag: "deu|!eng", _: Hallo}
18     - {name: world_str, tag: "eng", _: World}
19
20 #Operation
21 step:
22   name: say_hello
23   use: param_set #use existing parameterset

```

(continues on next page)

(continued from previous page)

24

```
do: echo '$hello_str $world_str' #shell command
```

The `tag` attribute and the `check_tags` tag allow you to define more complex boolean expressions. For example:

- `!` can be used for negation (`!deu` stands for not `deu`)
- `+` can be used as an AND operator to combine tag values (e.g. XML: `tag="!deu+eng"`; YAML: `tag: "!deu+eng"`).
- `|` can be used as an OR operator to combine tag values. (e.g. `tag1 | tag2` means that one of the two (or both) tags must be set.)
- `^` can be used as an XOR operator to combine tag values. (e.g. `tag1 ^ tag2` means that one of the two tags must be set, but not both.)
- Parentheses are also allowed

The `tag` attribute can be used within any `<element>` within the input file (except the `<jube>`). If several different `tag` attribute values are used in a script, they can be specified as a list separated by spaces from the command line.

To keep track of all tags, a description for each tag can be defined within the `<tags>`, where `<check_tags>` is also defined, using `<tag name="...">Description</tag>`. To force a description of all possible tags, the `forced` attribute can be set to `true`, so that an error is thrown if a tag is not documented.

All `<elements>` which contain a special `tag="..."` attribute will be hidden if the value of the tag evaluates to `false`. This means that JUBE will ignore the elements with these tags in its internal processing. Caution: This can lead to erroneous execution if you forget to set the necessary tags for execution, as JUBE will ignore e.g. a `<parameter>` provided with the corresponding `tag` attribute that evaluates to `false`.

Careful: This can lead to erroneous execution if you forget to set the necessary tags for execution, as JUBE will no longer consider e.g. parameters provided with the corresponding `tag` attribute.

To ensure that the user of the script specifies the necessary tag values that the script needs for successful execution, the `check_tag` element (added with JUBE version 2.6.0) can be used. It allows you to define tag values that must be specified when the script is called in order for it to run successfully. If none of the required `tag` combinations defined by `check_tag` are set by the user, an error message is displayed and the run is aborted.

In the example above, `check_tags: deu|eng` indicates that `deu` or `eng` must be set. When running the example using one of the specific `tag` values in `<check_tags>` (in this case `--tag eng`):

```
>>> jube run tagging.xml --tag eng
```

this results in the following output in the `stdout` file:

```
Hello World
```

3.8 Platform independent benchmarking

If you want to create platform independent benchmarks you can use the include features inside of *JUBE*.

All platform related sets must be declared in an includable file e.g. `platform.xml`. There can be multiple `platform.xml` in different directories to allow different platforms. By changing the `include-path` the benchmark changes its platform specific data.

An example benchmark structure is based on three include files:

- The main benchmark include file which contain all benchmark specific but platform independent data
- A mostly generic platform include file which contain benchmark independent but platform specific data (this can be created once and placed somewhere central on the system, it can be easily accessed using the `JUBE_INCLUDE_PATH` environment variable.
- A platform specific and benchmark specific include file which must be placed in a unique directory to allow `include-path` usage

Inside the `platform` directory you will find some example benchmark independent platform configuration files for the supercomputers at Forschungszentrum Jülich.

To avoid writing long include-paths every time you run a platform independent benchmark, you can store the include-path inside your input file. This can be mixed using the tagging-feature:

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <jube>
3   <include-path>
4     <path tag="plat1">some path</path>
5     <path tag="plat2">another path</path>
6     ...
7   </include-path>
8   ...
9 </jube>
```

Or in *YAML*:

```
1 ...
2 include-path:
3   path:
4     - {tag: plat1, _: "some path"}
5     - {tag: plat2, _: "another path"}
6 ...
```

Now you can run your benchmark using:

```
>>> jube run filename.xml --tag plat1
```

3.9 Multiple benchmarks

Often you only have one benchmark inside your input file. But it is also possible to store multiple benchmarks inside the same input file:

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <jube>
3   <benchmark name="a" outpath="bench_runs">...</benchmark>
4   <benchmark name="b" outpath="bench_runs">...</benchmark>
5   ...
6 </jube>
```

```
1 - name: a
2   # data for benchmark a
3 - name: b
4   # data for benchmark b
```

All benchmarks can use the same global (as a child of `<jube>`) declared sets. Often it might be better to use an include feature instead. *JUBE* will run every benchmark in the given order. Every benchmark gets a unique benchmark id.

To select only one benchmark you can use:

```
>>> jube run filename.xml --only-bench a
```

or:

```
>>> jube run filename.xml --not-bench b
```

This information can also be stored inside the input file:

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <jube>
3   <selection>
4     <only>a</only>
5     <not>b</not>
6   </selection>
7   ...
8 </jube>

```

3.10 Shared operations

Sometimes you want to communicate between the different workpackages of a single step or you want a single operation to run only once for all workpackages. Here you can use shared steps.

The files used for this example can be found inside `examples/shared`.

The input file `shared.xml`:

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <jube>
3   <benchmark name="shared" outpath="bench_run">
4     <comment>A shared folder example</comment>
5
6     <!-- Configuration -->
7     <parameterset name="param_set">
8       <parameter name="number" type="int">1,2,4</parameter>
9     </parameterset>
10
11    <!-- Operation -->
12    <step name="a_step" shared="shared">
13      <use>param_set</use>
14      <!-- shell command will run three times -->
15      <do>echo $jube_wp_id >> shared/all_ids</do>
16      <!-- shell command will run one time -->
17      <do shared="true">cat all_ids</do>
18    </step>
19  </benchmark>
20 </jube>

```

The input file `shared.yaml`:

```

1 name: shared
2 outpath: bench_run
3 comment: A shared folder example
4
5 #Configuration
6 parameterset:
7   name: param_set
8   parameter: {name: number, type: int, _: "1,2,4"}
9
10 #Operation
11 step:
12   name: a_step
13   shared: shared
14   use: param_set
15   do:
16     - echo $jube_wp_id >> shared/all_ids #shell command will run three times

```

(continues on next page)

(continued from previous page)

```
- {shared: true, _: cat all_ids} #shell command will run one times
```

The step must be marked using the `shared` attribute. The name, given inside this attribute, will be the name of a symbolic link, which will be created inside every single sandbox work directory pointing to a single shared folder. Every Workpackage can access this folder by using its own link. In this example every workpackage will write its own id into a shared file (`$jube_wp_id` is an internal variable, more of these you will find [here](#)).

To mark an operation to be a shared operation `shared="true"` inside the `<do>` must be used. The shared operation will start after all workpackages reached its execution position. The work directory for the shared operation is the shared folder itself.

You will get the following directory structure:

```
1 bench_run          # the given outputpath
2 |
3 +- 000000          # the benchmark id
4 |
5 +- configuration.xml # the stored benchmark configuration
6 +- workpackages.xml # workpackage information
7 +- 000000_a_step    # the first workpackage
8 |
9 +- done            # workpackage finished marker
10 +- work           # user sandbox folder
11 |
12 +- stderr         # standard error messages of used shell commands
13 +- stdout         # standard output of used shell commands
14 +- shared         # symbolic link pointing to shared folder
15 +- 000001_a_step  # workpackage information
16 +- 000002_a_step  # workpackage information
17 +- a_step_shared  # the shared folder
18 |
19 +- stdout         # standard output of used shell commands
20 +- stderr         # standard error messages of used shell commands
21 +- all_ids        # benchmark specific generated file
```

3.11 Environment handling

Shell environment handling can be very important to configure paths or parameter of your program.

The files used for this example can be found inside `examples/environment`.

The input file `environment.xml`:

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <jube>
3   <benchmark name="environment" outpath="bench_run">
4     <comment>An environment handling example</comment>
5
6     <!-- Configuration -->
7     <parameterset name="param_set">
8       <parameter name="EXPORT_ME" export="true">VALUE</parameter>
9     </parameterset>
10
11    <!-- Operations -->
12    <step name="first_step" export="true">
13      <do>export SHELL_VAR=Hello</do> <!-- export a Shell var -->
14      <do>echo "$$SHELL_VAR world"</do><!-- use exported Shell var -->
```

(continues on next page)

(continued from previous page)

```

15     </step>
16
17     <!-- Create a dependency between both steps -->
18     <step name="second_step" depend="first_step">
19         <use>param_set</use>
20         <do>echo $$EXPORT_ME</do>
21         <do>echo "$$SHELL_VAR again"</do> <!-- use exported Shell var out of previous_
↪ step -->
22     </step>
23 </benchmark>
24 </jube>

```

The input file environment.yaml:

```

1 name: environment
2 outpath: bench_run
3 comment: An environment handling example
4
5 #Configuration
6 parameterset:
7     name: param_set
8     parameter: {name: EXPORT_ME, export: true, _: VALUE}
9
10 step:
11     #Operation
12     - name: first_step
13       export: true
14       do:
15         - export SHELL_VAR=Hello #export a Shell var
16         - echo "$$SHELL_VAR world" #use exported Shell var
17
18     #Create a dependency between both steps
19     - name: second_step
20       depend: first_step
21       use: param_set
22       do:
23         - echo $$EXPORT_ME
24         - echo "$$SHELL_VAR again" #use exported Shell var out of previous step

```

In normal cases all <do> within one <step> shares the same environment. All **exported** variables of one <do> will be available inside the next <do> within the same <step>.

By using export="true" inside of a <parameter> you can export additional variables to your *Shell* environment. Be aware that this example uses \$\$ to explicitly use *Shell* substitution instead of *JUBE* substitution.

You can also export the complete environment of a step to a dependent step by using export="true" inside of <step>.

3.12 Parameter dependencies

Sometimes you need parameters which are based on other parameters or only a specific parameter combination makes sense and other combinations are useless or wrong. For this there are several techniques inside of *JUBE* to create such a more complex workflow.

The files used for this example can be found inside examples/parameter_dependencies.

The input file parameter_dependencies.xml:

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <jube>
3   <benchmark name="parameter_dependencies" outpath="bench_run">
4     <comment>A parameter_dependencies example</comment>
5
6     <!-- Configuration -->
7     <parameterset name="param_set">
8       <parameter name="index" type="int">0,1</parameter>
9       <parameter name="text" mode="python">["hello","world"][$index]</parameter>
10    </parameterset>
11
12    <parameterset name="depend_param_set0">
13      <parameter name="number" type="int">3,5</parameter>
14    </parameterset>
15
16    <parameterset name="depend_param_set1">
17      <parameter name="number" type="int">1,2,4</parameter>
18    </parameterset>
19
20    <!-- Operation -->
21    <step name="operation">
22      <use>param_set</use> <!-- use basic parameterset -->
23      <use>depend_param_set$index</use> <!-- use dependent parameterset -->
24      <use from="include_file.xml:depend_param_set0:depend_param_set1">
25        depend_param_set$index
26      </use>
27      <do>echo "$text $number $number2"</do>
28    </step>
29  </benchmark>
30 </jube>

```

The input file `parameter_dependencies.yaml`:

```

1 name: parameter_dependencies
2 outpath: bench_run
3 comment: A parameter_dependencies example
4
5 #Configuration
6 parameterset:
7   - name: param_set
8     parameter:
9       - {name: index, type: int, _: "0,1"} #comma separated integer must be in
↪ quotations
10      - {name: text, mode: python, _: '["hello","world"][$index]'} #attributes with "
↪ and [] must be in quotations
11      - name: depend_param_set0
12        parameter: {name: number, type: int, _: "3,5"} #comma separated integer must be
↪ in quotations
13      - name: depend_param_set1
14        parameter: {name: number, type: int, _: "1,2,4"} #comma separated integer must
↪ be in quotations
15
16 #Operation
17 step:
18   name: operation
19   use:
20     - param_set #use basic parameterset

```

(continues on next page)

(continued from previous page)

```

21 - depend_param_set$index #use dependent parameterset
22 - {from: 'include_file.yaml:depend_param_set0:depend_param_set1', _: depend_param_
    ↪set$index}
23 do: echo "$text $number $number2"

```

The include file include_file.xml:

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <jube>
3   <parameterset name="depend_param_set0">
4     <parameter name="number2" type="int">10</parameter>
5   </parameterset>
6
7   <parameterset name="depend_param_set1">
8     <parameter name="number2" type="int">20</parameter>
9   </parameterset>
10 </jube>

```

The include file include_file.yaml:

```

1 parameterset:
2 - name: depend_param_set0
3   parameter: {name: number2, type: int, _: 10}
4 - name: depend_param_set1
5   parameter: {name: number2, type: int, _: 20}

```

The easiest way to handle dependencies is to define an index-parameter which can be used in other scripting parameters to combine all dependent parameter combinations.

Also complete sets can be marked as dependent towards a specific parameter by using this parameter in the `<use>`-tag. When using parametersets out of another file the correct set-name must be given within the `from` attribute, because these sets will be loaded in a pre-processing step before the corresponding parameter will be evaluated. Also sets out of different files can be combined within the same `<use>` by using the `file1:set1,file2:set2` syntax. The sets names must be unique.

3.13 Parameter update

Once a parameter is specified and evaluated the first time, its value will not change. Sometimes this behaviour can produce the wrong behaviour:

```
<parameter name="foo">$jube_wp_id</parameter>
```

In this example `foo` should hold the `$jube_wp_id`. If you have two steps, where one step depends on the other one `foo` will be available in both, but it will only be evaluated in the first one.

There is a simple workaround to change the update behaviour of a parameter by using the attribute `update_mode`:

- `update_mode="never"` No update (default behaviour)
- `update_mode="use"` Re-evaluate the parameter if the parameterset is explicitly used
- `update_mode="step"` Re-evaluate the parameter for each new step
- `update_mode="cycle"` Re-evaluate the parameter for each new cycleloop, but not at the begin of a new step
- `update_mode="always"` Combine step and cycle

Within a cycle loop no new workpackages can be created. Templates will be reevaluated, but they can not increase the number of existing workpackages within a cycle.

Within the result generation, the parameter value, which is presented in the result table is the value of the selected analysed step. If another parameter representation is needed as well, all other steps can be reached by using `<parameter_name>_<step_name>`.

The files used for this example can be found inside `examples/parameter_update`.

The input file `parameter_update.xml`:

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <jube>
3   <benchmark name="parameter_updates" outpath="bench_run">
4     <comment>A parameter_update example</comment>
5
6     <!-- Configuration -->
7     <parameterset name="foo">
8       <parameter name="bar_never" mode="text" update_mode="never">
9         iter_never: $jube_wp_id
10      </parameter>
11      <parameter name="bar_use" mode="text" update_mode="use">
12        iter_use: $jube_wp_id
13      </parameter>
14      <parameter name="bar_step" mode="text" update_mode="step">
15        iter_step: $jube_wp_id
16      </parameter>
17    </parameterset>
18
19    <!-- Operation -->
20    <step name="step1">
21      <use>foo</use>
22      <do>echo $bar_never</do>
23      <do>echo $bar_use</do>
24      <do>echo $bar_step</do>
25    </step>
26
27    <step name="step2" depend="step1">
28      <use>foo</use>
29      <do>echo $bar_never</do>
30      <do>echo $bar_use</do>
31      <do>echo $bar_step</do>
32    </step>
33
34    <step name="step3" depend="step2">
35      <do>echo $bar_never</do>
36      <do>echo $bar_use</do>
37      <do>echo $bar_step</do>
38    </step>
39  </benchmark>
40 </jube>

```

The input file `parameter_update.yaml`:

```

1 name: parameter_updates
2 outpath: bench_run
3 comment: A parameter_update example
4
5 #Configuration
6 parameterset:
7   name: foo
8   parameter:

```

(continues on next page)

(continued from previous page)

```

9   - {name: bar_never, mode: text, update_mode: never, _: "iter_never: $jube_wp_id"}
10  - {name: bar_use, mode: text, update_mode: use, _: "iter_use: $jube_wp_id"}
11  - {name: bar_step, mode: text, update_mode: step, _: "iter_step: $jube_wp_id"}
12
13  #Operation
14  step:
15    - name: step1
16      use: foo
17      do:
18        - echo $bar_never
19        - echo $bar_use
20        - echo $bar_step
21    - name: step2
22      depend: step1
23      use: foo
24      do:
25        - echo $bar_never
26        - echo $bar_use
27        - echo $bar_step
28    - name: step3
29      depend: step2
30      do:
31        - echo $bar_never
32        - echo $bar_use
33        - echo $bar_step

```

The use and influence of the three update modes `update_mode="never"`, `update_mode="use"` and `update_mode="step"` is shown here. Keep in mind, that the steps have to be dependent from each other leading to identical outputs otherwise.

3.14 Step iteration

Especially in the context of benchmarking, an application should be executed multiple times to generate some meaningful statistical values. The handling of statistical values is described in *Statistic pattern values*. This allows you to aggregate multiple result lines if your application automatically support to run multiple times.

In addition there is also an iteration feature within JUBE to run a specific step and its parametrisation multiple times.

The files used for this example can be found inside `examples/iterations`.

The input file `iterations.xml`:

```

1  <?xml version="1.0" encoding="UTF-8"?>
2  <jube>
3    <benchmark name="iterations" outpath="bench_run">
4      <comment>A Iteration example</comment>
5
6      <!-- Configuration -->
7      <parameterset name="param_set">
8        <parameter name="foo" type="int">1,2,4</parameter>
9        <parameter name="bar" mode="text" update_mode="step">$foo iter:$jube_wp_
10 iteration</parameter>
11      </parameterset>
12
13      <step name="first_step" iterations="2">
14        <use>param_set</use> <!-- use existing parameterset -->

```

(continues on next page)

(continued from previous page)

```

14     <do>echo $bar</do> <!-- shell command -->
15 </step>
16
17 <step name="second_step" depend="first_step" iterations="2">
18     <do>echo $bar</do> <!-- shell command -->
19 </step>
20
21 <!-- analyse without reduce -->
22 <analyser name="analyse_no_reduce" reduce="false">
23     <analyse step="second_step" />
24 </analyser>
25
26 <!-- Analyse with reduce -->
27 <analyser name="analyse" reduce="true">
28     <analyse step="second_step" />
29 </analyser>
30
31 <result>
32     <use>analyse</use>
33     <use>analyse_no_reduce</use>
34     <table name="result" style="pretty">
35         <column>jube_res_analyser</column>
36         <column>jube_wp_id_first_step</column>
37         <column>jube_wp_id</column>
38         <column>jube_wp_iteration_first_step</column>
39         <column>jube_wp_iteration</column>
40         <column>foo</column>
41     </table>
42 </result>
43 </benchmark>
44 </jube>

```

The input file iterations.yaml:

```

1 name: iterations
2 outpath: bench_run
3 comment: A Iteration example
4
5 #Configuration
6 parameterset:
7     name: param_set
8     parameter:
9         - {name: foo, type: int, _: "1,2,4"}
10        - {name: bar, update_mode: step, _: '$foo iter:$jube_wp_iteration'}
11
12 step:
13     - name: first_step
14       iterations: 2
15       use: param_set #use existing parameterset
16       do: echo $bar #shell command
17     - name: second_step
18       depend: first_step
19       iterations: 2
20       do: echo $bar #shell command
21
22 analyser:

```

(continues on next page)

(continued from previous page)

```

23 #analyse without reduce
24 - name: analyse_no_reduce
25   reduce: false
26   analyse:
27     step: second_step
28 #analyse with reduce
29 - name: analyse
30   reduce: true
31   analyse:
32     step: second_step
33
34 result:
35   use: [analyse, analyse_no_reduce]
36   table:
37     name: result
38     style: pretty
39     column:
40       - jube_res_analyser
41       - jube_wp_id_first_step
42       - jube_wp_id
43       - jube_wp_iteration_first_step
44       - jube_wp_iteration
45       - foo

```

In this example, both steps 1 and 2 are executed 2 times for each parameter and dependency configuration. Because of the given parameter, step 1 is executed 6 times in total (3 parameter combinations x 2). Step 2 is executed 12 times (6 from the dependent step x 2). Each run will be executed in the normal way using its individual sandbox folder.

\$jube_wp_iteration holds the individual iteration id. The update_mode is needed here to reevaluate the parameter bar in step 2.

In the analyser reduce=true or reduce=false can be enabled, to allow you to see all individual results or to aggregate all results of the same parameter combination. for the given step. If reduce=true is enabled (the default behaviour) the output of the individual runs, which uses the same parametrisation, are treated like a big continuous file before applying the statistical patterns.

```

1 | jube_res_analyser | jube_wp_id_first_step | jube_wp_id | jube_wp_iteration_first_
  ↳ step | jube_wp_iteration | foo |
2 |-----|-----|-----|-----|
  ↳ --|-----|-----|
3 | analyse_no_reduce |          0 |          6 |
  ↳ 0 |          0 |          1 |
4 | analyse_no_reduce |          0 |          7 |
  ↳ 0 |          1 |          1 |
5 | analyse_no_reduce |          1 |          8 |
  ↳ 1 |          2 |          1 |
6 | analyse_no_reduce |          1 |          9 |
  ↳ 1 |          3 |          1 |
7 | analyse_no_reduce |          2 |         10 |
  ↳ 0 |          0 |          2 |
8 | analyse_no_reduce |          2 |         11 |
  ↳ 0 |          1 |          2 |
9 | analyse_no_reduce |          3 |         12 |
  ↳ 1 |          2 |          2 |
10 | analyse_no_reduce |          3 |         13 |
    ↳ 1 |          3 |          2 |

```

(continues on next page)

(continued from previous page)

11	analyse_no_reduce			4	14	┌
	↪ 0	0	4			
12	analyse_no_reduce			4	15	┌
	↪ 0	1	4			
13	analyse_no_reduce			5	16	┌
	↪ 1	2	4			
14	analyse_no_reduce			5	17	┌
	↪ 1	3	4			
15	analyse			5	16	┌
	↪ 1	2	4			
16	analyse			0	7	┌
	↪ 0	1	1			
17	analyse			1	8	┌
	↪ 1	2	1			
18	analyse			2	10	┌
	↪ 0	0	2			
19	analyse			3	12	┌
	↪ 1	2	2			
20	analyse			4	15	┌
	↪ 0	1	4			

3.15 Step cycle

Instead of having a new workpackage you can also redo the <do> commands inside a step using the cycle-feature. In contrast to the iterations, all executions for the cycle feature take place inside the same folder.

The files used for this example can be found inside `examples/cycle`.

The input file `cycle.xml`:

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <jube>
3   <benchmark name="cycle" outpath="bench_run">
4     <comment>A cycle example</comment>
5
6     <step name="a_step" cycles="5">
7       <do break_file="done">echo $jube_wp_cycle</do>
8       <do active="$jube_wp_cycle==2">touch done</do>
9     </step>
10
11   </benchmark>
12 </jube>

```

The input file `cycle.yaml`:

```

1 ---
2 benchmark:
3   - name: cycle
4     outpath: bench_run
5     comment: A cycle example
6
7   step:
8     - name: a_step
9       cycles: 5
10      do:
11        - _: echo $jube_wp_cycle

```

(continues on next page)

(continued from previous page)

```

12         break_file: done
13     - _: touch done
14         active: $jube_wp_cycle==2
15

```

The `cycles` attribute allows to repeat all `<do>` commands within a step multiple times. The `break_file` can be used to cancel the loop and all following commands in the current cycle (the command itself is still executed). In the given example the output will be:

```

0
1
2
3

```

In contrast to the iterations, all executions for the cycle feature take place inside of the same folder.

3.16 Parallel workpackages

In a standard `jube run` a queue is filled with workpackages and then processed in serial. To enable parallel execution of independent workpackages, which belong to the expansions of a step, the argument `procs` of `<step>` can be used.

The files used for this example can be found inside `examples/parallel_workpackages`.

The input file `parallel_workpackages.xml`:

```

1  <?xml version="1.0" encoding="UTF-8"?>
2  <jube>
3    <benchmark name="parallel_workpackages" outpath="bench_run">
4      <comment>A parallel workpackages demo</comment>
5
6      <parameterset name="param_set">
7        <parameter name="i" type="int" mode="python">",".join([ str(i) for i in range(0,
8      ↪ 10)])</parameter>
9      </parameterset>
10
11     <step name="parallel_execution" suffix="{i}" procs="4">
12       <use>param_set</use>
13       <do>echo "{i}"</do>
14       <do>N=1000000 ; a=1; k=0; while [ "$k" -lt $N ]; do echo $(( 2*k + 1 + $a )) ;
15       ↪ k=$(( k + 1 )) ; done</do>
16     </step>
17   </benchmark>
18 </jube>

```

The input file `parallel_workpackages.yaml`:

```

1  name: parallel_workpackages
2  outpath: bench_run
3  comment: A parallel workpackages demo
4
5  parameterset:
6    name: param_set
7    parameter: {name: i, type: int, mode: python, _: "\"",\".join([ str(i) for i in
8  ↪ range(0,10)])"}
9  step:
10    name: parallel_execution

```

(continues on next page)

(continued from previous page)

```

10  suffix: ${i}
11  procs: 4
12  use: param_set
13  do:
14    - "echo \"${i}\""
15    - "N=1000000 ; a=1; k=0; while [ \"$k\" -lt $N ]; do echo $(( 2*k + 1 + $a )) ; k=
    ↪ $(( k + 1 )) ; done"

```

In the example above the expansion of the parameter `i` will lead to the creation of 10 workpackages of the step `parallel_execution`. Due to the given argument `procs="4"` JUBE will start 4 worker processes which will distribute the execution of the workpackages among themselves. `N` within the JUBE script represents the number of computation iterations to simulate a computational workload at hand. The parameters `N`, `procs` and the upper bound of range within this prototypical example can be alternated to study runtime, memory usage and load of CPUs.

Important hints:

- `<do shared="true">` is not supported if `procs` is set for the corresponding step.
- If `<step shared="...">` is set, then the user is responsible to avoid data races within the shared directory.
- Switching to an alternative `work_dir` for a step can also lead to data races if all expansions of the step access the same `work_dir`. Recommendation: Don't use a shared `work_dir` in combination with `procs`.
- This feature is implemented based on the Python package `multiprocessing` and doesn't support inter-node communication. That's why the parallelisation is limited to a single shared memory compute node.
- Be considerate when working on a multi-user system with shared resources. The parallel feature of JUBE can easily exploit a whole compute node.
- Parallel execution of a JUBE script can lead to much higher memory demand compared to serial execution with `procs=1`. In this case it is advised to reduce `procs` leading to reduced memory usage.

3.17 Result database

Results can also be stored into a database to simplify result management.

The files used for this example can be found inside `examples/result_database`.

The input file `result_database.xml`:

```

1  <?xml version="1.0" encoding="UTF-8"?>
2  <jube>
3    <benchmark name="result_database" outpath="bench_run">
4      <comment>result database creation</comment>
5
6      <parameterset name="param_set">
7        <parameter name="number" type="int">1,2,4</parameter>
8      </parameterset>
9
10     <patternset name="pattern">
11       <pattern name="number_pat" type="int">Number: $jube_pat_int</pattern>
12     </patternset>
13
14     <step name="write_number">
15       <use>param_set</use>
16       <do>echo "Number: $number"</do>
17     </step>
18
19     <analyser name="analyse">

```

(continues on next page)

(continued from previous page)

```

20     <use>pattern</use>
21     <analyse step="write_number">
22         <file>stdout</file>
23     </analyse>
24 </analyser>
25
26 <result>
27     <use>analyse</use>
28     <!-- creating a database containing the columns "number" and "number_pat" -->
29     <!-- one table of the name "results" is created within the database -->
30     <!-- optionally, you can use the "file" attribute to specify an alternative_
↪ storage location for the database -->
31     <database name="results" primekeys="NUM">
32         <key primekey="true">number</key>
33         <key title="NUM">number_pat</key>
34     </database>
35 </result>
36 </benchmark>
37 </jube>

```

The input file result_database.yaml:

```

1  name: result_database
2  outpath: bench_run
3  comment: result database creation
4
5  parameterset:
6      name: param_set
7      parameter: {name: number, type: int, _: "1,2,4"}
8
9  patternset:
10     name: pattern
11     pattern: {name: number_pat, type: int, _: "Number: $jube_pat_int"}
12
13  step:
14     name: write_number
15     use: param_set
16     do: "echo \"Number: $number\""
17
18  analyser:
19     name: analyse
20     use: pattern
21     analyse:
22         step: write_number
23         file: stdout
24
25  result:
26     use: analyse
27     database:
28         # creating a database containing the columns "number" and "number_pat"
29         # one table of the name "results" is created within the database
30         # optionally, you can use the "file" attribute to specify an alternative storage_
↪ location for the database
31         name: results
32         primekeys: "NUM"
33         key:

```

(continues on next page)

(continued from previous page)

```

34 - {primekey: true, _: number}
35 - {title: NUM, _: number_pat}

```

The default database will be located as follows and has the database tag name, which is here `results`, as root name concatenated with the appendix `.dat`:

```

1 bench_run
2 |
3 +- 0000000
4 |
5 +- result
6 |
7 +- results.dat

```

The database tag takes the argument `name`. `name` is also the name of the table created within a database. If `sqlite3` is installed the contents of the database can be shown with the following command line.

```

1 >>> sqlite3 -header -table bench_run/0000000/result/results.dat 'SELECT * FROM results'
2 +-----+-----+
3 | number | number_pat |
4 +-----+-----+
5 | 1      | 1          |
6 | 2      | 2          |
7 | 4      | 4          |
8 +-----+-----+

```

The key tag adds columns to the database table having the same type as the corresponding parameter or pattern. Information of columns of the database table `results` can be shown as follows.

```

1 >>> sqlite3 -header -table bench_run/0000000/result/results.dat 'PRAGMA table_
2 info(results)'
3 +-----+-----+-----+-----+-----+-----+
4 | cid | name      | type | notnull | dflt_value | pk |
5 +-----+-----+-----+-----+-----+-----+
6 | 0   | number    | int  | 0       |             | 1 |
7 | 1   | number_pat | int  | 0       |             | 2 |

```

The `file` argument takes a relative (to the current working directory) or absolute path to an alternative/user-defined location for the database file. Assuming that `file="result_database.dat"` was set in the above example, a file named `result_database.dat` would be created in the current working directory where `jube result` was invoked, containing a database named `results`, and the file `bench_run/0000000/result/results.dat` would no longer contain the database, but the path specified in the `file` attributes.

Invoking `jube result` a second time will update the database specified by the `file` parameter. Without the `primekey` attribute of the `key`-tag, three additional rows would be added to the `results` table which are identical to the already existing three rows. Setting the `primekey` attribute of the `key`-tag to `true` adds the key to the `primekeys`, ensuring that a new row is only added to the database table if the column values specified in the `primekeys` do not match exactly in the database table. In this example, no new rows would be added. The use of the database attribute `primekeys` is deprecated. Instead, use the `primekey` attribute of the `key`. Updating `primekeys` is not supported.

To have a look into a database within a python script the python modules `sqlalchemy` or `pandas` can be used.

3.18 Creating a do log

To increase reproducibility of the do statements within a workpackage of a step and to archive the environment during execution, a do log can be printed. A do log tries to mimic an executable script recreating the environment at execution time. The files used for this example can be found inside `examples/do_log`.

The input file `do_log.xml`:

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <jube>
3   <benchmark name="do_log_example" outpath="bench_run">
4
5     <parameterset name="param_set">
6       <parameter name="number">1,2,3,4,5</parameter>
7     </parameterset>
8
9     <step name="execute" shared="shared" do_log_file="do_log">
10      <use>param_set</use>
11      <do>cp ../../../../loreipsum${number} shared</do>
12      <do shared="true">grep -r -l "Hidden!" loreipsum*</do>
13    </step>
14
15  </benchmark>
16 </jube>

```

The input file `do_log.yaml`:

```

1 - name: do_log_example
2   outpath: bench_run
3
4   parameterset:
5     - name: "param_set"
6       parameter:
7         - {name: "number", _: "1,2,3,4,5"}
8
9   step:
10     name: execute
11     use:
12       - param_set
13     do_log_file: "do_log"
14     shared: "shared"
15     do:
16       - cp ../../../../loreipsum${number} shared
17       - {shared: "true", _: "grep -r -l \"Hidden!\" loreipsum*"}

```

In this example a hidden string is searched for within 5 files and the name of the file containing the hidden string is printed.

For the initial execution of this example within `bench_run/000000/00000[0-4]_execute` each can be found a `do_log` file. These files can be executed manually by prefixing it with `/bin/sh`. The scripts will reproduce the environment at execution time, the execution and the result output. Keep in mind that the shared `grep` will be executed by the benchmark with id 4 only.

3.19 The duplicate option

To simplify advanced tagging and parameter concatenation the `duplicate` option within parametersets or parameters can be stated.

The input file `duplicate.xml`:

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <jube>
3   <benchmark name="parameter_duplicate_example" outpath="bench_run">
4     <comment>parameter duplicate example</comment>
5
6     <parameterset name="options" duplicate="concat">
7       <parameter name="iterations" >1</parameter>
8       <parameter name="iterations" tag="few" >2,3,4</parameter>
9       <parameter name="iterations" tag="many" >20,30,40</parameter>
10    </parameterset>
11
12    <parameterset name="result">
13      <parameter name="sum" mode="python">int(${iterations}*(${iterations}+1)/2)</
14    </parameter>
15    </parameterset>
16
17    <step name="perform_iterations">
18      <use>options,result</use>
19      <do>echo $sum</do>
20    </step>
21
22  </benchmark>
23 </jube>

```

The input file duplicate.yaml:

```

1 name: parameter_duplicate_example
2 outpath: bench_run
3 comment: parameter duplicate example
4
5 parameterset:
6   - name: options
7     duplicate: concat
8     parameter:
9       - {name: iterations, _: "1"}
10      - {name: iterations, tag: few, _: "2,3,4"}
11      - {name: iterations, tag: many, _: "20,30,40"}
12   - name: result
13     parameter:
14       - {name: sum, mode: "python", _: "int(${iterations}*(${iterations}+1)/2)"}
15
16 step:
17   name: perform_iterations
18   use: "options,result"
19   do: "echo $sum"

```

In this example the `duplicate` option with the value `concat` is stated for a `parameterset`. This leads to a concatenation of parameter values of the same name. In combination with the tagging option for parameters the user can specify which options are included into the parameters. If the user states the tags `few` and `many` the parameter `iterations` takes the values `1,2,3,4,20,30,40`.

The default option of `duplicate` for `parametersets` is `replace` which leads to a replacing of parameters if they are mentioned more than once. A third option for the `duplicate` option for `parametersets` is `error`. In this case the execution is aborted if a parameter is defined more than once.

The option `duplicate` can also be stated for parameters. In this case the parameters `duplicate` option is prioritized over the `parametersets` one. The possible values for parameters `duplicate` option are `none`, `replace`, `concat` and `error`. `none` is the default value and leads to the `duplicate` option being ignored for this parameter such that the `parametersets` `duplicate` option is taking precedence. The other three options have the same effect

as in the parameterset.

FREQUENTLY ASKED QUESTIONS

4.1 Parameter groups

Within *JUBE* you can define parameter groups to allow only specific parameter combinations.

E.g. you have two parameters:

```
<parameter name="foo">10,100</parameter>
<parameter name="bar">20,200</parameter>
```

```
parameter:
- { name: foo, _: '10,100' }
- { name: bar, _: '20,200' }
```

Without any additional change, *JUBE* will run four parameter combinations (foo=10,bar=20, foo=100, bar=20, foo=10,bar=200, foo=100,bar=200). But maybe within your configuration only foo=10,bar=20 and foo=100,bar=200 make sense. For this you can use the parameter dependencies feature and small *Python* snippets (*Parameter dependencies*) to split the four combinations into two groups, by using a dummy index value:

```
<parameter name="i">0,1</parameter>
<parameter name="foo" mode="python">[10,100][$i]</parameter>
<parameter name="bar" mode="python">[20,200][$i]</parameter>
```

```
parameter:
- { name: i, _: '0,1' }
- { name: foo, mode: python, _: '[10,100][$i]' }
- { name: bar, mode: python, _: '[20,200][$i]' }
```

Instead of using a numerical index, you can also use a string value for selection:

```
<parameter name="key">tick,tock</parameter>
<parameter name="foo" mode="python">
  {"tick" : 10,
   "tock" : 100}["${key}"]
</parameter>
<parameter name="bar" mode="python">
  {"tick" : 20,
   "tock" : 200}["${key}"]
</parameter>
```

```
parameter:
- { name: key, _: 'tick,tock' }
- name: foo
  mode: python
  _ : |
```

(continues on next page)

(continued from previous page)

```

    {
      "tick" : 10,
      "tock" : 100
    }["${key}"]
- name: bar
  mode: python
  -: |
    {
      "tick" : 20,
      "tock" : 200
    }["${key}"]

```

Also default values are possible:

```

<parameter name="foo" mode="python">
  {"tick" : 10,
   "tock" : 100}.get("${key}",0)
</parameter>

```

```

parameter:
- name: foo
  mode: python
  -: |
    {
      "tick" : 10,
      "tock" : 100
    }.get("${key}",0)

```

4.2 Workdir change

Sometimes you want to execute a step outside of the normal *JUBE* directory structure. This can be done by using the `work_dir`-attribute inside the `<step>`-tag. If you use the `work_dir` *JUBE* does not create a unique directory structure. So you have to create this structure on your own if you need unique directories e.g. by using the *jube_variables*.

```

<step name="a_step" work_dir="path_to_dir/${jube_benchmark_papid}/${jube_wp_papid}_${jube_step_name}">
  ...
</step>

```

```

step:
  name: a_step
  work_dir: "bench_run/${jube_benchmark_papid}/${jube_wp_papid}_${jube_step_name}"

```

Using the `*_papid` variables will help to create a sorted directory structure.

JUBE does not create any symbolic links inside the changed work directories. If you want to access files, out of a dependend step, you can use a `<fileset>` and the `rel_path_ref`-attribute.

```

<fileset name="needed_files">
  <link rel_path_ref="internal">dependent_step_name/a_file</link>
</files>

```

```

fileset:
  name: needed_files

```

(continues on next page)

(continued from previous page)

link:

```
- {rel_path_ref: internal, _: dependent_step_name/a_file}
```

This will create a link inside your alternative working dir and the link target path will be seen relative towards the original *JUBE* directory structure. So here you can use the normal automatic created link to access all dependend files.

To access files out of an alternative working directory in a following step and if you created this working directory by using the *jube_variables*, you can use `jube_wp_parent_<parent_name>_id` to get the id of the parent step to use it within a path definition.

4.3 XML character handling

The *JUBE XML* based input format bases on the general *XML* rules. Here some hints for typical *XML* problems:

Linebreaks are not allowed inside a tag-option (e.g. `<sub ... dest="...\n...">` is not possible). Inside a tag multiple lines are no problem (e.g. inside of `<parameter>...</parameter>`). Often multiple lines are also needed inside a `<sub>`. Linebreaks are possible for the `dest=""` part, by switching to the alternative `<sub>` syntax:

```
<sub source="...">
...
</sub>
```

Whitespaces will only be removed in the beginning and in the end of the whole string. So indentation of a multiline string can create some problems.

Some characters are not allowed inside an *XML* script or at least not inside a tag-option. Here are some of the typical replacments:

- `<`: `<`;
- `>`: `>`;
- `&`: `&`;
- `"`: `"`;
- `'`: `'`;

4.4 YAML character handling

The *JUBE YAML* based input format bases on the general *YAML* rules.

Instead of tags in the *XML* format the *YAML* format uses keys which values are a list of elements or other keys.

The files used for this example can be found inside `examples/yaml`.

The input file `hello_world.yaml`:

```
benchmark: # having only a single benchmark, this key is optional
  name: hello_world
  outpath: bench_run
  comment: A simple hello world in yaml

#Configuration
parameterset:
  name: hello_parameter
  parameter: {name: hello_str, _: Hello World}

#Operation
step:
```

(continues on next page)

(continued from previous page)

```

name: say_hello
use: hello_parameter # special key _ can be skipped
do:
  - _: echo $hello_str # - is optional in this case, as there is only one do entry
active: true

```

You can use different styles of writing key value pairs: In the example, the `parameter` is declared in one line using `{}`. Multiple key value pairs can be stored per element. The main content attribute is marked by using `_`. As an alternative you can write the key value pairs amongst multiple lines using the same indent as the preceding line, like the key `do` in the example. If a key like `use` has only a value, you can write it in one line without using the special `_` key.

A list of elements can be specified by using `[]` or by using `-` amongst multiple lines (always keeping the same indent).

YAML also has a number of special characters which can be integrated by using quotation marks:

The input file `special_values.yaml`:

```

name: special values
outpath: bench_run
comment: An example for values that need to be in quotations

parameterset:
  name: special_parameters
  parameter:
    - {name: integer, type: int, _: "1,2,4"} #comma separated values need to be
    ↪quoted
    - {name: "NUMBER", _: "#3"} #values with # need to be quoted

patternset:
  name: special_pattern
  pattern:
    - {name: result, type: int, _: "Result: test"} #values with : need to be quoted
    - {name: integers, type: int, _: "Integers = {$integer}"} #values with {} need to
    ↪be quoted
    - {name: integer, type: int, _: "'Integer' = $NUMBER"} #values with ' need to be
    ↪quoted

```

Anytime you have a symbol like `#`, `'`, `,`, `:` or `{}` you have to enclose the entire value in quotation marks.

4.5 Analyse multiple output files

This FAQ entry is only relevant for *JUBE* versions prior version 2.2. Since version 2.2 *JUBE* automatically creates a combined result table.

Within an `<analyser>` you can analyse multiple files. Each `<analyser>` `<analyse>` combination will create independent result entries:

```

<analyser name="analyse">
  <use>a_patternset</use>
  <analyse step="step_A">
    <file>stdout</file>
  </analyse>
  <analyse step="step_B">
    <file>stdout</file>
  </analyse>
</analyser>

```

In this example the `<patternset>` `a_patternset` will be used for both files. This is ok if there are only patterns which match either the `step_A` stdout file or the `step_B` stdout file.

If you want to use a file dependent patternset you can move the use to a `<file>` attribute instead:

```
<analyser name="analyse">
  <analyse step="step_A">
    <file use="a_patternset_A">stdout</file>
  </analyse>
  <analyse step="step_B">
    <file use="a_patternset_B">stdout</file>
  </analyse>
</analyser>
```

This avoids the generation of incorrect result entries. A `from=...` option is not available in this case. Instead you can copy the patternset first to your local file by using the `init_with` attribute.

Due to the independent `result_entries`, you will end up with the following result table if you mix the extracted pattern:

pattern1_of_A	pattern2_of_A	pattern1_of_B
1	A	
2	B	
		10
		11
		12
		13

The different `<analyse>` were not combined. So you end up with independent result lines for each workpackage. *JUBE* does not see possible step dependencies in this point the user has to set the dependencies manually:

```
<analyser name="analyse">
  <analyse step="step_B">
    <file use="a_patternset_B">stdout</file>
    <file use="a_patternset_A">step_A/stdout</file>
  </analyse>
</analyser>
```

Now we only have one `<analyse>` and we are using the autogenerated link to access the dependent step. This will create the correct result:

pattern1_of_A	pattern2_of_A	pattern1_of_B
1	A	10
2	B	11
1	A	12
2	B	13

4.6 Extract data from a specific text block

In many cases the standard program output is structured into multiple blocks:

```
blockA:
...
time=20

blockB:
...
time=30
```

Using a simple `<pattern>` like `time=$jube_pat_int` will match all `time=` lines (the default match will be the first one, and *Statistic pattern values* are available as well). However in many cases a specific value from a specific block should be extracted. This is possible by using `\s` within the pattern for each individual newline character within the block, or by using the `dotall` option:

```
<pattern name="a_pattern" dotall="true">blockB:.*?time=$jube_pat_int</pattern>
```

pattern:

```
- {name: a_pattern, dotall: true, _: 'blockB:.*?time=$jube_pat_int'}
```

This only extracts 30 from blockB. Setting `dotall="true"` allows to use the `.` to take care of all newline characters in between (by default newline characters are not matched by `.`).

4.7 Restart a workpackage execution

If a problem occurs outside of the general *JUBE* handling (e.g. a crashed HPC job or a broken dependency) it might be necessary to restart a specific workpackage. *JUBE* allows this restart by removing the problematic workpackage entry and using the `jube continue` command afterwards:

```
jube remove bechmark_directory --id <id> --workpackage <workpackage_id>
...
jube continue bechmark_directory
```

This will rerun the specific workpackage. The *JUBE* configuration will stay unchanged. It is not possible to change the `<paramter>` or `<step>` configuration later on. Shared `<do>` operations (`shared=true`) will be ignored within such a rerun scenario except if all workpackages of a specific step were removed and the full step is re-executed.

COMMAND LINE DOCUMENTATION

Here you will find a list of all available *JUBE* command line options. You can also use:

```
jube -h
```

to get a list of all available commands.

Because of the *shell* parsing mechanism take care if you write your optional arguments after the command name before the positional arguments. You **must** use `--` to split the ending of an optional (if the optional argument takes multiple input elements) and the start of the positional argument.

When using *BASH* you can use the `jube` complete mechanism to enable a command line autocompletion.

5.1 general

General commandline options (can also be used in front of a subcommand)

```
jube [-h] [-V] [-v] [--debug] [--force] [--strict] [--devel] {...}
```

- h, --help**
show general help information
- V, --version**
show version information
- v, --verbose**
enable verbose console output (use `-vv` to show stdout during execution and `-vvv` to show log and stdout)
- debug**
use debugging mode (no shell script execution)
- force**
ignore any *JUBE* version conflict
- strict**
force strict *JUBE* version check
- devel**
developer mode (show complete error messages)

5.2 run

Run a new benchmark.

```
1 jube run [-h] [--only-bench ONLY_BENCH [ONLY_BENCH ...]]
2           [--not-bench NOT_BENCH [NOT_BENCH ...]] [-t TAG [TAG ...]]
3           [--hide-animation] [--include-path INCLUDE_PATH [INCLUDE_PATH ...]]
4           [-o OUTPATH] [-a] [-r] [-e]
5           [-m COMMENT] [--id ID [ID ...]] FILE [FILE ...]
```

-h, --help
show command help information

--only-bench ONLY_BENCH [ONLY_BENCH ...]
only run specific benchmarks given by benchmark name

--not-bench NOT_BENCH [NOT_BENCH ...]
do not run specific benchmarks given by benchmark name

-t TAG [TAG ...], --tag TAG [TAG ...]
use specific tags when running this file. This will be used for *tagging*

--hide-animation
hide the progress bar animation (if you want to use *JUBE* inside a scripting environment)

--include-path INCLUDE_PATH [INCLUDE_PATH ...]
add additional include paths where to search for include files

-a, --analyse
run analyse after finishing run command

-r, --result
run result after finishing run command (this will also start analyse)

-e, --exit
run will exit if there is an error

-m COMMENT, --comment COMMENT
overwrite benchmark specific comment

-o OUTPATH, --outpath OUTPATH
overwrite outpath directory (relative to the execution location)

-i ID [ID ...], --id ID [ID ...]
use specific benchmark id (must be ≥ 0)

FILE [FILE ...]
input *XML* file

5.3 continue

Continue an existing benchmark.

```
jube continue [-h] [-i ID [ID ...]] [--hide-animation] [-a] [-r] [-e] [DIRECTORY]
```

-h, --help
show command help information

-i ID [ID ...], --id ID [ID ...]
select benchmark id, negative ids count backwards from the end; default: last found benchmark inside the benchmark directory; special ids *all* or *last* can be used

--hide-animation
hide the progress bar animation (if you want to use *JUBE* inside a scripting environment)

-a, --analyse
run analyse after finishing run command

-r, --result
run result after finishing run command (this will also start analyse)

-e, --exit
run will exit if there is an error

DIRECTORY
directory which contains benchmarks, default: .

5.4 analyse

Run the analyse procedure.

```
jube analyse [-h] [-i ID [ID ...]] [-u UPDATE_FILE]
              [--include-path INCLUDE_PATH [INCLUDE_PATH ...]]
              [-t TAG [TAG ...]] [DIRECTORY]
```

- h, --help**
show command help information
- i ID [ID ...], --id ID [ID ...]**
select benchmark id, negative ids count backwards from the end; default: last found benchmark inside the benchmark directory; special ids *all* or *last* can be used
- u UPDATE_FILE, --update UPDATE_FILE**
use given input *XML* file to update patternsets, analyser and result before running the analyse
- include-path INCLUDE_PATH [INCLUDE_PATH ...]**
add additional include paths where to search for include files (when using --update)
- t TAG [TAG ...], --tag TAG [TAG ...]**
use specific tags when running this file. This will be used for *tagging* (when using --update)
- DIRECTORY**
directory which contains benchmarks, default: .

5.5 result

Run the result creation.

```
jube result [-h] [-i ID [ID ...]] [-a] [-r] [-u UPDATE_FILE] [-n NUM]
             [-s {pretty, csv, aligned}] [--include-path INCLUDE_PATH [INCLUDE_PATH ...]]
             [-t TAG [TAG ...]] [-o RESULT_NAME [RESULT_NAME ...]]
             [--select SELECT [SELECT ...]] [--exclude EXCLUDE [EXCLUDE ...]]
             ↪ [DIRECTORY]
```

- h, --help**
show command help information
- i ID [ID ...], --id ID [ID ...]**
select benchmark id, negative ids count backwards from the end; default: last found benchmark inside the benchmark directory; special ids *all* or *last* can be used
- a, --analyse**
run analyse before running result command
- r, --reverse**
reverse benchmark output order when multiple benchmarks are given
- n, --num**
show only last N benchmarks
- u UPDATE_FILE, --update UPDATE_FILE**
use given input *XML* file to update patternsets, analyser and result before running the analyse
- s {pretty, csv, aligned}, --style {pretty, csv, aligned}**
overwrites table style type
- include-path INCLUDE_PATH [INCLUDE_PATH ...]**
add additional include paths where to search for include files (when using --update)
- t TAG [TAG ...], --tag TAG [TAG ...]**
use specific tags when running this file. This will be used for *tagging* (when using --update)

- o RESULT_NAME [RESULT_NAME ...], -only RESULT_NAME [RESULT_NAME ...]**
only create specific results given by name
- select SELECT [SELECT ...]**
display only given columns from the result (changes also the output to the result file)
- exclude EXCLUDE [EXCLUDE ...]**
excludes given columns from the result (changes also the output to the result file)

DIRECTORY

directory which contains benchmarks, default: .

5.6 output

Shows path and content of the stdout and stderr files of the given benchmark.

```
jube output [-h] [-i ID [ID ...]] [-s STEP [STEP ...]] [-w WORKPACKAGE [WORKPACKAGE ..  
→.]]  
           [-d] [-o {stdout,stderr}] [DIRECTORY]
```

- h, --help**
show command help information
- i ID [ID ...], --id ID [ID ...]**
select benchmark id, negative ids count backwards from the end; default: last found benchmark inside the benchmark directory; special ids *all* or *last* can be used
- s STEP [STEP ...], --step STEP [STEP ...]**
show filenames for given step
- w WORKPACKAGE [WORKPACKAGE ...], --workpackage WORKPACKAGE [WORKPACKAGE ...]**
show filenames for given workpackages id
- d, --display**
display content of output file
- o {stdout,stderr}, --only {stdout,stderr}**
show only stdout or stderr

DIRECTORY

directory which contains benchmarks, default: .

5.7 comment

Add or manipulate the benchmark comment.

```
jube comment [-h] [-i ID [ID ...]] [-a] comment [DIRECTORY]
```

- h, --help**
show command help information
- i ID [ID ...], --id ID [ID ...]**
select benchmark id, negative ids count backwards from the end; default: last found benchmark inside the benchmark directory; special ids *all* or *last* can be used
- a, --append**
append new comment instead of overwrite existing one

comment

new comment

DIRECTORY

directory which contains benchmarks, default: .

5.8 remove

Remove an existing benchmark

```
jube remove [-h] [-i ID [ID ...]] [-w WORKPACKAGE [WORKPACKAGE ...]] [-f] [DIRECTORY]
```

-h, --help

show command help information

-i ID [ID ...], --id ID [ID ...]

select benchmark id, negative ids count backwards from the end; default: last found benchmark inside the benchmark directory; special ids *all* or *last* can be used

-w WORKPACKAGE [WORKPACKAGE ...], --workpackage WORKPACKAGE [WORKPACKAGE ...]

specific workpackage id to be removed

-f, --force

do not prompt

DIRECTORY

directory which contains benchmarks, default: .

5.9 info

Get benchmark specific information

```
jube info [-h] [-i ID [ID ...]] [-s STEP [STEP ...]] [-p] [-c [SEPARATOR]] [-w WORKPACKAGE [WORKPACKAGE ...]] [DIRECTORY]
```

-h, --help

show command help information

-i ID [ID ...], --id ID [ID ...]

show benchmark specific information

-s STEP [STEP ...], --step STEP [STEP ...]

show step specific information

-c [SEPARATOR], --csv-parametrization [SEPARATOR]

display only parametrization of given step using *csv* format, *csv* separator is optional

-p, --parametrization

display only parametrization of given step

-w WORKPACKAGE [WORKPACKAGE ...], --workpackage WORKPACKAGE [WORKPACKAGE ...]

show workpackage specific information by given id

DIRECTORY

show directory specific information

5.10 log

Show logs for benchmark

```
jube log [-h] [-i ID [ID ...]] [-c COMMAND [COMMAND ...]] [DIRECTORY]
```

-h, --help

show command help information

-i ID [ID ...], --id ID [ID ...]

select benchmark id, negative ids count backwards from the end; default: last found benchmark inside the benchmark directory; special ids *all* or *last* can be used

-c COMMAND [COMMAND ...], --command COMMAND [COMMAND ...]
show only logs for specified commands

DIRECTORY
directory which contains benchmarks, default: .

5.11 status

Show benchmark status RUNNING or FINISHED.

```
jube status [-h] [-i ID [ID ...]] [DIRECTORY]
```

-h, --help
show command help information

-i ID [ID ...], --id ID [ID ...]
select benchmark id, negative ids count backwards from the end; default: last found benchmark inside the benchmark directory; special ids *all* or *last* can be used

DIRECTORY
directory which contains benchmarks, default: .

5.12 tag

Show tags description

```
jube tag [-h] [-i ID [ID ...]] [PATH]
```

-h, --help
show command help information

-i ID [ID ...], --id ID [ID ...]
select benchmark id, negative ids count backwards from the end; default: last found benchmark inside the benchmark directory; special ids *all* or *last* can be used

PATH
path to input file or directory which contains benchmarks, default: .

5.13 complete

Generate shell completion. Usage: eval "\$(jube complete)"

```
jube complete [-h] [--command-name COMMAND_NAME]
```

-h, --help
show command help information

--command-name COMMAND_NAME, -c COMMAND_NAME
name of command to be complete, default: program name which was used to run the `complete` command

5.14 help

Command help

```
jube help [-h] [command]
```

-h, --help
show command help information

command

command to get help about

5.15 update

Check *JUBE* version

```
jube update [-h]
```

-h, --help

show command help information

GLOSSARY

analyse

Analyse an existing benchmark. The analyser will scan through all files given inside the configuration by using the given patternsets.

If no benchmark id is given, last benchmark found in directory will be used. If benchmark directory is missing, current directory will be used.

analyser_tag

The analyser describe the steps and files which should be scanned using a set of pattern.

```
<analyser name="..." reduce="...">
  <use from="">...</use>
  ...
  <analyse step="...">
    <file use="">...</file>
  </analyse>
  ...
</analyser>
```

- you can use different patternsets to analyse a set of files
- only patternsets are usable
- using patternsets `<use>set1,set2</use>` is the same as `<use>set1</use><use>set2</use>`
- the from-attribute is optional and can be used to specify an external set source
- any name must be unique, it is not allowed to reuse a set
- the step-attribute contains an existing stepname
- each file using each workpackage will be scanned separately
- the use argument inside the `<file>` tag is optional and can be used to specify a file specific patternset;
 - the global `<use>` and this local use will be combined and evaluated at the same time
 - a `from` subargument is not possible in this local use
- `reduce` is optional (default: `true`)
 - `true` : Combine result lines if iteration-option is used
 - `false` : Create single line for each iteration

benchmark_tag

The main benchmark definition

```
<benchmark name="..." outpath="...">
...
</benchmark>
```

- container for all benchmark information

- benchmark-name must be unique inside input file
- outpath contains the path to the root folder for benchmark runs
 - multiple benchmarks can use the same folder
 - every benchmark and every (new) run will create a new folder (named by an unique benchmark id) inside this given outpath
 - the path will be relative to input file location

check_tags_tag

Specify combination of tags that must be set.

```
<check_tags>...</check_tags>
```

- The combination is set using boolean algebra.
- For the logical operation ‘conjunction’, i.e. ‘and’, the sign + is used. Example: tag1 + tag2 means that both tags must be set.
- For the logical operation ‘disjunction’, i.e. ‘or’, the character | is used. Example: tag1 | tag2 means that one of the two or both tags must be set.
- For the logical operation ‘exclusive disjunction’, i.e. ‘xor’, the character ^ is used. Example: tag1 ^ tag2 means that one of the two (not both!) tags must be set.
- In addition, the character ! can be used for the logical operation ‘negation’, i.e. ‘not’. Example: !tag1 means that the tag tag1 must not be set.

column_tag

A line within a ASCII result table. The <column>-tag can contain the name of a pattern or the name of a parameter.

```
<column colw="..." format="..." title="...">...</column>
```

- colw is optional: column width
- title is optional: column title
- format can contain a C like format string: e.g. format=".2f"

comment

Add or manipulate the comment string.

If no benchmark id is given, last benchmark found in directory will be used. If benchmark directory is missing, current directory will be used.

comment_tag

Add a benchmark specific comment. These comment will be stored inside the benchmark directory.

```
<comment>...</comment>
```

complete

Generate shell completion.

continue

Continue an existing benchmark. Not finished steps will be continued, if they are leaving pending mode.

If no benchmark id is given, last benchmark found in directory will be used. If benchmark directory is missing, current directory will be used.

copy_tag

A copy can be used to copy a file or directory from your normal filesystem to your sandbox work directory.

```
<copy source_dir="..." target_dir="..." name="..." rel_path_ref="..." separator=  
→"..." active="...">...</copy>
```

- `source_dir` is optional, will be used as a prefix for the source filenames
- `target_dir` is optional, will be used as a prefix for the target filenames
- `name` is optional, it can be used to rename the file inside your work directory (will be ignored if you use shell extensions in your pathname)
- `rel_path_ref` is optional
 - `external` or `internal` can be chosen, default: `external`
 - `external`: rel-paths based on position of xml-file
 - `internal`: rel-paths based on current work directory (e.g. to link files of another step)
- `active` is optional
 - can be set to `true` or `false` or any *Python* parsable bool expression to enable or disable the single command
 - *parameter* are allowed inside this attribute
- each copy-tag can contain a list of filenames (or directories), separated by `,`, the default separator can be changed by using the `separator` attribute
 - if `name` is present, the lists must have the same length
- you can copy all files inside a directory by using `directory/*`
 - this cannot be mixed using `name`
- in the execution step the given files or directories will be copied

database_tag

Create sqlite3 database

```
<database name="..." primekeys="..." file="..." filter="...">
  <key title="..." primekey="...">...</key>
  ...
</database>
```

- `name`: name of the table in the database
- Unlike the result table, the unit attribute of a parameter or pattern is not taken into account.
- `primekeys` is optional: can contain a list of parameter or pattern names (separated by `,`). Given parameters or patterns will be used as primary keys of the database table. All primekeys have to be listed as a `<key>` as well and if the `title` attribute of a `key` is set, then the value of the `title` attribute must be used in the `primekeys` attribute (and not the parameter or pattern name). Modification of primary keys of an existing table is not supported. If no primekeys are set then each *jube result* will add new rows to the database. Otherwise rows with matching primekeys will be updated. **Important note: The use of the ``database`` attribute ``primekeys`` is deprecated and will be removed soon. Instead, use the ``primekey`` attribute of the ``key``.**
- `file` is optional. The given value should hold the full path to the database file. If the file including the path does not exist it will be created. Absolute and relative paths are supported.
- `filter` is optional. It can contain a bool expression to show only specific result entries.
- `<key>` can be specified in the database result and must contain an single parameter or pattern name.
 - `title` is optional: alternative key title. Used to define a custom database column name.
 - `primekey` is optional: If `primekey` is set to `true`, the key is added to the database primekeys. (default: `false`) The `primekeys` attribute of the database-tag is deprecated and will be removed.

directory_structure

- every (new) benchmark run will create its own directory structure
- every single workpackage will create its own directory structure

- user can add files (or links) to the workpackage dir, but the real position in filesystem will be seen as a blackbox
- general directory structure:

```
benchmark_runs (given by "outpath" in xml-file)
|
+- 0000000 (determined through benchmark-id)
  |
  +- 0000000_compile (step: just an example, can be arbitrary chosen)
    |
    +- work (user environment)
    +- done (workpackage finished information file)
    +- ... (more jube internal information files)
  +- 0000001_execute
    |
    +- work
      |
      +- compile -> ../../0000000_compile/work (automatic generated link,
↳ for depending step)
    +- wp_done_00 (single "do" finished, but not the whole workpackage)
    +- ...
  +- 0000002_execute
  +- result (result data)
  +- configuration.xml (benchmark configuration information file)
  +- workpackages.xml (workpackage graph information file)
  +- analyse.xml (analyse data)
+- 0000001 (determined through benchmark-id)
  |
  +- 0000000_compile (step: just an example, can be arbitrary chosen)
  +- 0000001_execute
  +- 0000002_postprocessing
```

do_tag

A do contain a executable *Shell* operation.

```
<do stdout="..." stderr="..." active="...">...</do>
<do done_file="..." error_file="...">...</do>
<do break_file="...">...</do>
<do shared="true">...</do>
<do work_dir="...">...</do>
```

- do can contain any *Shell*-syntax-snippet (*parameter* will be replaced ... `$nameofparameter` ...)
- stdout- and stderr-filename are optional (default: stdout and stderr)
- work_dir is optional, it can be used to change the work directory of this single command (relatively seen towards the original work directory)
- active is optional
 - can be set to true or false or any *Python* parsable bool expression to enable or disable the single command
 - *parameter* are allowed inside this attribute
- done_file-filename and error_file are optional
 - by using done_file the user can mark async-steps. The operation will stop until the script will create the named file inside the work directory.
 - by using error_file the operation will produce a error if the named file can be found inside the work directory. This feature can be used together with the done_file to signalise broken

async-steps.

- `break_file-filename` is optional
 - by using `break_file` the user can stop further cycle runs. the current step will be directly marked with finalized and further `<do>` will be ignored.
- `shared="true"`
 - can be used inside a step using a shared folder
 - cmd will be **executed inside the shared folder**
 - cmd will run once (synchronize all workpackages)
 - `$jube_wp_...` - parameter cannot be used inside the shared command

fileset_tag

A fileset is a container to store a bundle of links and copy commands.

```
<fileset name="..." init_with="...">
  <link>...</link>
  <copy>...</copy>
  <prepare>...</prepare>
  ...
</fileset>
```

- `init_with` is optional
 - if the given filepath can be found inside of the `JUBE_INCLUDE_PATH` and if it contains a fileset using the given name, all link and copy will be copied to the local set
 - the name of the external set can differ to the local one by using `init-with="filename.xml:external_name"`
- link and copy can be mixed within one fileset (or left)
- filesets can be used inside the step-command

general_structure_xml

```
<?xml version="1.0" encoding="UTF-8"?>
<!-- Basic top level JUBE structure -->
<jube>
  <!-- optional additional include paths -->
  <include-path>
    <path>...</path>
    ...
  </include-path>
  <!-- optional benchmark selection -->
  <selection>
    <only>...</only>
    <not>...</not>
    <tag>...</tag>
    ...
  </selection>
  <!-- optional must-have tag specification -->
  <check_tags>...</check_tags>
  <!-- global sets -->
  <parameterset name="">...</parameterset>
  <substitutionset name="">...</substitutionset>
  <fileset name="">...</fileset>
  <patternset name="">...</patternset>
  ...
</jube>
```

(continues on next page)

(continued from previous page)

```

<benchmark name="" outpath="">
  <!-- optional benchmark comment -->
  <comment>...</comment>
  <!-- local benchmark parametersets -->
  <parameterset name="">...</parameterset>
  ...
  <!-- files, which should be used -->
  <fileset name="">...</fileset>
  ...
  <!-- substitution rules -->
  <substituteset name="">...</substituteset>
  ...
  <!-- pattern -->
  <patternset name="">...</patternset>
  ...
  <!-- commands -->
  <step name="">...</step>
  ...
  <!-- analyse -->
  <analyser name="">...</analyser>
  ...
  <!-- result -->
  <result>...</result>
  ...
</benchmark>
...
</jube>

```

general_structure_yaml

```

# optional additional include paths
include-path:
  ...

# optional benchmark selection
selection:
  only: ...
  not: ...
  tag: ...

# optional must-have tag specification
check_tags:
  ...

# global sets
parameterset:
  ...
substitutionset:
  ...
fileset:
  ...
patternset:
  ...

benchmark: # can be skipped if only a single benchmark is handled
- name: ...

```

(continues on next page)

(continued from previous page)

```

outpath: ...
# optional benchmark comment
comment: ...

# local sets
parameterset:
    ...
substitutionset:
    ...
fileset:
    ...
patternset:
    ...

# commands
step:
    ...

analyser:
    ...
result:
    ...

```

include-path_tag

Add some include paths where to search for include files.

```

<include-path>
  <path>...</path>
  ...
</include-path>

```

- the additional path will be scanned for include files

include_tag

Include *XML*-data from an external file.

```
<include from="..." path="..." />
```

- `<include>` can be used to include an external *XML*-structure into the current file
- can be used at every position (inside the `<jube>`-tag)
- path is optional and can be used to give an alternative xml-path inside the include-file (default: root-node)

info

Show info for the given benchmark directory, a given benchmark or a specific step.

If benchmark directory is missing, current directory will be used.

iofile_tag

A iofile declare the name (and path) of a file used for substitution.

```
<iofile in="..." out="..." out_mode="..." />
```

- in and out filepath are relative to the current work directory for every single step (not relative to the path of the inputfile)
- in and out can be the same
- out_mode is optional, can be w or a (default: w)

- w : out-file will be overridden
- a : out-file will be appended

jube_pattern

List of available jube pattern:

- \$jube_pat_int: integer number
- \$jube_pat_nint: integer number, skip
- \$jube_pat_fp: floating point number
- \$jube_pat_nfp: floating point number, skip
- \$jube_pat_wrd: word
- \$jube_pat_nwrđ: word, skip
- \$jube_pat_bl: blank space (variable length), skip

jube_variables

List of available jube variables:

- Benchmark:
 - \$jube_benchmark_name: current benchmark name
 - \$jube_benchmark_id: current benchmark id
 - \$jube_benchmark_padiđ: current benchmark id with preceding zeros
 - \$jube_benchmark_home: original input file location
 - \$jube_benchmark_rundir: main benchmark specific execution directory
 - \$jube_benchmark_start: benchmark starting time
- Step:
 - \$jube_step_name: current step name
 - \$jube_step_iterations: number of step iterations (default: 1)
 - \$jube_step_cycles: number of step cycles (default: 1)
- Workpackage:
 - \$jube_wp_id: current workpackage id
 - \$jube_wp_padiđ: current workpackage id with preceding zeros
 - \$jube_wp_status: current workpackage status [RUNNING, DONE, ERROR]
 - \$jube_wp_iteration: current iteration number (default: 0)
 - \$jube_wp_parent_<parent_name>_id: workpackage id of selected parent step
 - \$jube_wp_relpath: relative path to workpackage work directory (relative towards configuration file)
 - \$jube_wp_abspath: absolute path to workpackage work directory
 - \$jube_wp_envstr: a string containing all exported parameter in shell syntax:

```
export par=$par
export par2=$par2
```
 - \$jube_wp_envlist: list of all exported parameter names
 - \$jube_wp_cycle: id of current step cycle (starts at 0)

link_tag

A link can be used to create a symbolic link from your sandbox work directory to a file or directory inside your normal filesystem.

```
<link source_dir="..." target_dir="..." name="..." rel_path_ref="..." separator=
→ "..." active="...">...</link>
```

- `source_dir` is optional, will be used as a prefix for the source filenames
- `target_dir` is optional, will be used as a prefix for the target filenames
- `name` is optional, it can be used to rename the file inside your work directory (will be ignored if you use shell extensions in your pathname)
- `rel_path_ref` is optional
 - `external` or `internal` can be chosen, default: `external`
 - `external`: rel-paths based on position of xml-file
 - `internal`: rel-paths based on current work directory (e.g. to link files of another step)
- `active` is optional
 - can be set to `true` or `false` or any *Python* parsable bool expression to enable or disable the single command
 - *parameter* are allowed inside this attribute
- each link-tag can contain a list of filenames (or directories), separated by `,`, the default separator can be changed by using the `separator` attribute
 - if `name` is present, the lists must have the same length
- in the execution step the given files or directories will be linked

log

Show logs for the given benchmark directory or a given benchmark.

If no benchmark id is given, last benchmark found in directory will be used. If benchmark directory is missing, current directory will be used.

output

Shows path and content of the stdout and stderr files of the given benchmark.

If no benchmark id is given, last benchmark found in will be used.

parameter_space

The parameter space for a specific benchmark run is the bundle of all possible parameter combinations. E.g. there are two different parameters: `a = 1,2` and `b = "p","q"` then you will get four different parameter combinations: `a=1, b="p"`; `a=1, b="q"`; `a=2, b="p"`; `a=2, b="q"`.

The parameter space of a specific step will be one of these parameter combinations. To fulfill all combinations the step will be executed multiple times (each time using a new combination). The specific combination of a step and an expanded parameter space is named *workpackage*.

parameter_tag

A parameter can be used to store benchmark configuration data. A set of different parameters will create a specific parameter environment (also called *parameter space*) for the different steps of the benchmark.

```
<parameter name="..." mode="..." type="..." separator="..." export="..." unit="..
→ ." update_mode="..." duplicate="...">...</parameter>
```

- a parameter can be seen as variable: Name is the name to use the variable, and the text between the tags will be the real content
- name must be unique inside the given parameterset
- `type` is optional (only used for sorting, default: `string`)

- `mode` is optional (used for script-types, default: `text`)
- `separator` is optional, default: `,`
- `export` is optional, if set to `true` the parameter will be exported to the shell environment when using `<do>`
- `unit` is optional, will be used in the result table
- if the text contains the given (or the implicit) separator, a template will be created
- use of another parameter:
 - inside the parameter definition, a parameter can be reused: `... $nameofparameter ...`
 - the parameter will be replaced multiple times (to handle complex parameter structures; max: 5 times)
 - the substitution will be run before the execution step starts with the current *parameter space*. Only parameters reachable in this step will be usable for substitution!
- Scripting modes allowed:
 - `mode="python"`: allow *Python* snippets (using `eval <cmd>`)
 - `mode="perl"`: allow *Perl* snippets (using `perl -e "print <cmd>"`)
 - `mode="shell"`: allow *Shell* snippets
 - `mode="env"`: include the content of an available environment variable
 - `mode="tag"`: include the tag name if the tag was set during execution, otherwise the content is empty
- Templates can be created, using scripting e.g.: `", ".join([str(2**i) for i in range(3)])`
- `update_mode` is optional (default: `never`)
 - can be set to `never`, `use`, `step`, `cycle` and `always`
 - depending on the setting the parameter will be reevaluated:
 - * `never`: no reevaluation, even if the parameterset is used multiple times
 - * `use`: reevaluation if the parameterset is explicitly used
 - * `step`: reevaluation in each new step
 - * `cycle`: reevaluation in each cycle, but not at the begin of a new step (number of workpackages will stay unchanged)
 - * `always`: reevaluation in each step and cycle
- `duplicate` is optional and of relevance, if there are more than one parameter definitions with the same name within one parameterset. This `duplicate` option has higher priority than the `duplicate` option of the parameterset. `duplicate` must contain one of the following four options:
 - `none` (default): The `duplicate` option of the parameterset is prioritized
 - `replace`: Parameters with the same name are overwritten
 - `concat`: Parameters with the same name are concatenated
 - `error`: Throws an error, if parameters with the same name are defined

parameterset_tag

A parameterset is a container to store a bundle of *parameters*.

```
<parameterset name="..." init_with="..." duplicate="...">
  <parameter>...</parameter>
  ...
</parameterset>
```

- parameterset-name must be unique (cannot be reused inside substitutionsets or filesets)
- `init_with` is optional
 - if the given filepath can be found inside of the `JUBE_INCLUDE_PATH` and if it contains a parameterset using the given name, all parameters will be copied to the local set
 - local parameters will overwrite imported parameters
 - the name of the external set can differ to the local one by using `init-with="filename.xml:external_name"`
- parametersets can be used inside the step-command
- parametersets can be combined inside the step-tag, but they must be compatible:
 - Two parametersets are compatible if the parameter intersection (given by the parameter-name), only contains parameter based on the same definition
 - These two sets are compatible:

```
<parameterset name="set1">
  <parameter name="test">1,2,4</parameter>
  <parameter name="test2">foo</parameter>
</parameterset>
<parameterset name="set2">
  <parameter name="test">1,2,4</parameter>
  <parameter name="test3">bar</parameter>
</parameterset>
```

- These two sets are not compatible:

```
<parameterset name="set1">
  <parameter name="test">1,2,4</parameter>
  <parameter name="test2">foo</parameter>
</parameterset>
<parameterset name="set2">
  <parameter name="test">2</parameter> <!-- Template in set1 -->
  <parameter name="test2">bar</parameter> <!-- Other content in set2 -->
</parameterset>
```

- `duplicate` is optional and of relevance, if there are more than one parameter definitions with the same name within one parameterset. This `duplicate` option has lower priority than the `duplicate` option of the parameters. `duplicate` must contain one of the following three options:
 - `replace` (default): Parameters with the same name are overwritten
 - `concat`: Parameters with the same name are concatenated
 - `error`: Throws an error, if parameters with the same name are defined

pattern_tag

A pattern is used to parse your output files and create your result data.

```
<pattern name="..." default="..." unit="..." mode="..." type="..." dotall="...">
  ...</pattern>
```

- `unit` is optional, will be used in the result table
- `mode` is optional, allowed modes:
 - `pattern`: a regular expression (default)
 - `text`: simple text and variable concatenation
 - `perl`: snippet evaluation (using *Perl*)

- `python`: snippet evaluation (using *Python*)
- `shell`: snippet evaluation (using *Shell*)
- `type` is optional, specify datatype (for sort operation)
 - default: `string`
 - allowed: `int`, `float` or `string`
- `default` is optional: Specify default value if pattern cannot be found or if it cannot be evaluated
- `dotall` is optional (default: `false`): Can be set to `true` or `false` to specify if a `.` within the regular expression should also match newline characters, which can be very helpfull to extract a line only after a specific header was mentioned.

patternset_tag

A patternset is a container to store a bundle of patterns.

```
<patternset name="..." init_with="...">
  <pattern>...</pattern>
  ...
</patternset>
```

- patternset-name must be unique
- `init_with` is optional
 - if the given filepath can be found inside of the `JUBE_INCLUDE_PATH` and if it contains a patternset using the given name, all pattern will be copied to the local set
 - local pattern will overwrite imported pattern
 - the name of the external set can differ to the local one by using `init-with="filename.xml:external_name"`
- patternsets can be used inside the analyser tag
- different sets, which are used inside the same analyser, must be compatible

prepare_tag

The prepare can contain any *Shell* command you want. It will be executed like a normal `<do>` inside the step where the corresponding fileset is used. The only difference towards the normal `do` is, that it will be executed **before** the substitution will be executed.

```
<prepare stdout="..." stderr="..." work_dir="..." active="...">...</prepare>
```

- `stdout`- and `stderr`-filename are optional (default: `stdout` and `stderr`)
- `work_dir` is optional, it can be used to change the work directory of this single command (relatively seen towards the original work directory)
- `active` is optional
 - can be set to `true` or `false` or any *Python* parsable bool expression to enable or disable the single command
 - *parameter* are allowed inside this attribute

remove

The given benchmark will be removed.

If no benchmark id is given, last benchmark found in directory will be removed.

Only the *JUBE* internal directory structure will be deleted. External files and directories will stay unchanged.

If no benchmark id is given, last benchmark found in directory will be used. If benchmark directory is missing, current directory will be used.

result

Create a result table.

If no benchmark id is given, last benchmark found in directory will be used. If multiple benchmarks are selected (e.g. by using `--id all`), a combined result view of all available benchmarks in the given directory will be created. If benchmark directory is missing, current directory will be used.

result_tag

The result tag is used to handle different visualisation types of your analysed data.

```
<result result_dir="...">
  <use>...</use>
  ...
  <table>...</table>
  <syslog>...</syslog>
  <database>...</database>
  ...
</result>
```

- `result_dir` is optional. Here you can specify an different output directory. Inside of this directory a subfolder named by the current benchmark id will be created. Default: `benchmark_dir/result`
- only analyser are usable
- using analyser `<use>set1,set2</use>` is the same as `<use>set1</use><use>set2</use>`

run

Start a new benchmark run by parsing the given *JUBE* input file.

selection_tag

Specify tags or select benchmarks by name.

```
<selection>
  <only>...</only>
  <not>...</not>
  <tag>...</tag>
  ...
</selection>
```

- multiple `<tag>` are allowed to specify tags (see tagging)
- select or unselect a benchmark by name
- only selected benchmarks will run (when using the `run` command)
- multiple `<only>` and `<not>` are allowed
- `<only>` and `<not>` can contain a name list divided by ,

statistical_values

If there are multiple pattern matches within one file, multiple files or when using multiple iterations. *JUBE* will create some statistical values automatically:

- `first`: first match (default)
- `last`: last match
- `min`: min value
- `max`: max value
- `avg`: average value
- `std`: standard deviation
- `sum`: sum
- `cnt`: counter

These variables can be accessed within the the result creation or to create derived pattern by `variable_name_<statistic_option>` e.g. `${nodes_min}`

The variable name itself always matches the first match.

status

Show status string (RUNNING or FINISHED) for the given benchmark.

If no benchmark id is given, last benchmark found in directory will be used. If benchmark directory is missing, current directory will be used.

step_tag

A step give a list of *Shell* operations and a corresponding parameter environment.

```
<step name="..." depend="..." work_dir="..." suffix="..." shared="..." active="..."
→."
    export="..." max_async="..." iterations="..." cycles="..." procs="..." do_
→log_file="...">
    <use from="">...</use>
    ...
    <do></do>
    ...
</step>
```

- parametersets, filesets and substitutionsets are usable
- using sets `<use>set1,set2</use>` is the same as `<use>set1</use><use>set2</use>`
- parameter can be used inside the `<use>`-tag
- the `from` attribute is optional and can be used to specify an external set source
- any name must be unique, it is **not allowed to reuse** a set
- `depend` is optional and can contain a list of other step names which must be executed before the current step
- `max_async` is optional and can contain a number (or a parameter) which describe how many *workpackages* can be executed asynchronously (default: 0 means no limitation). This option is only important if a *do* inside the step contains a `done_file` attribute and should be executed in the background (or managed by a jobsystem). In this case *JUBE* will manage that there will not be to many instances at the same time. To update the benchmark and start further instances, if the first ones were finished, the *continue* command must be used.
- `work_dir` is optional and can be used to switch to an alternative work directory
 - the user had to handle **uniqueness of this directory** by his own
 - no automatic parent/children link creation
- `suffix` is optional and can contain a string (parameters are allowed) which will be attached to the default workpackage directory name
- `active` is optional
 - can be set to `true` or `false` or any *Python* parsable bool expression to enable or disable the single command
 - *parameter* are allowed inside this attribute
- `shared` is optional and can be used to create a shared folder which can be accessed by all workpackages based on this step
 - a link, named by the attribute content, is used to access the shared folder
 - the shared folder link will not be automatically created in an alternative working directory!
- `export="true"`

- the environment of the current step will be exported to an dependent step
- `iterations` is optional. All workpackages within this step will be executed multiple times if the `iterations` value is used.
- `cycles` is optional. All `<do>` commands within the step will be executed `cycles`-times
- `procs` is optional. Amount of processes used to execute the parameter expansions of the corresponding step in parallel.
- `do_log_file` is optional. Name or path of a do log file trying to mimick the do steps and the environment of a workpacakge of a step to produce an executable script.

sub_tag

A substitution expression.

```
<sub source="..." dest="..." mode="..." />
```

- `source`-string will be replaced by `dest`-string
- both can contain parameter: ... `$nameofparameter` ...
- `mode` is optional (default: `text`). Can be used to switch between `text` and `regex` substitution

substituteset_tag

A substituteset is a container to store a bundle of *sub* commands.

```
<substituteset name="..." init_with="...">
  <iofile/>
  ...
  <sub/>
  ...
</substituteset>
```

- `init_with` is optional
 - if the given filepath can be found inside of the `JUBE_INCLUDE_PATH` and if it contains a substituteset using the given name, all `iofile` and `sub` will be copied to the local set
 - local `iofile` will overwrite imported ones based on `out`, local `sub` will overwrite imported ones based on `source`
 - the name of the external set can differ to the local one by using `init-with="filename.xml:external_name"`
- substitutesets can be used inside the step-command

syslog_tag

A syslog result type

```
<syslog name="..." address="..." host="..." port="..." sort="..." format="..."
  ↪filter="...">
  <key format="..." title="...">...</key>
  ...
</syslog>
```

- Syslog daemon can be given by a `host` and `port` combination (default port: 541) or by a socket address e.g.: `/dev/log` (mixing of `host` and `address` is not allowed)
- `format` is optional: can contain a log format written in a pythonic way (default: `jube[%(process)s]: %(message)s`)
- `sort` is optional: can contain a list of parameter- or patternnames (separated by `,`). Given patterntype or parametertype will be used for sorting
- `filter` is optional, it can contain a bool expression to show only specific result entries

- `<key>` can be specified in the syslog result and must contain an single parameter or pattern name.
 - `format` can contain a C like format string: e.g. `format=".2f"`
 - `title` is optional: alternative key title.
- Unlike the result table, the unit attribute of a parameter or pattern is not taken into account.

table_tag

A simple ASCII based table ouput.

```
<table name="..." style="..." sort="..." separator="..." transpose="..." filter=
→ "...">
  <column>...</column>
  ...
</table>
```

- `style` is optional; allowed styles: `csv`, `pretty`, `aligned`; default: `csv`
- `separator` is optional; only used in `csv`-style, default: `,`
- `sort` is optional: can contain a list of parameter- or patternnames (separated by `,`). Given `pattern`type or `parameter`type will be used for sorting
- `<column>` must contain an single parameter- or patternname
- `transpose` is optional (default: `false`)
- `filter` is optional, it can contain a bool expression to show only specific result entries

tag

Show tag description for the given input file, the given benchmark directory or a specific benchmark.

If path to input file or benchmark directory is missing, current directory will be used. If no benchmark id is given, last benchmark found in directory will be used.

tagging

Tagging is a simple way to mark parts of your input file to be includable or excludable.

- Every available `<tag>` (not the root `<jube>`-tag) can contain a tag-attribute
- The tag-attribute can contain a list of names: `tag="a,b,c"` or “not” names: `tag="a,!b,c"`
- When running *JUBE*, multiple tags can be send to the input-file parser:

```
jube run <filename> --tag a b
```

- `<tags>` which does not contain one of these names will be hidden inside the include file
- `<tags>` which does not contain any tag-attribute will stay inside the include file
- “not” tags are more important than normal tags: `tag="a,!b,c"` and running with `a b` will hide the `<tag>` because the `!b` is more important than the `a`

tags_tag

Specify tag description and combination of tags that must be set.

```
<tags forced="...">
  <check_tags>...</check_tags>
  <tag name="...">...</tag>
</tags>
```

- `forced` is optional, if it is set to `true`, you will be forced to describe every possible tag. (default: `false`)
- multiple `<check_tags>` and `<tag>` are allowed
- In the `<tag>`, you can write a description for the tag with the given name.

types

Parameter and *Pattern* allow a type specification. This type is either used for sorting within the result table and is also used to validate the parameter content. The types are not used to convert parameter values, e.g. a floating value will stay unchanged when used in any other context even if the type int was specified.

allowed types are:

- `string` (this is also the default type)
- `int`
- `float`

update

Check if a newer JUBE version is available.

update_mode

The update mode is parameter attribute which can be used to control the reevaluation of the parameter content.

These update modes are available:

- `never`: no reevaluation, even if the parameterset is used multiple times
- `use`: reevaluation if the parameterset is explicitly used
- `step`: reevaluation in each new step
- `cycle`: reevaluation in each cycle, but not at the begin of a new step (number of workpackages will stay unchanged)
- `always`: reevaluation in each step and cycle

workpackage

A workpackage is the combination of a *step* (which contains all operations) and one parameter setting out of the expanded *parameter space*.

Every workpackage will run inside its own sandbox directory!

A

advanced tutorial, 16
analyse, 12, 58, 65
analyse multiple files, 54
analyser_tag, 65

B

benchmark_tag, 65

C

check_tags_tag, 66
column_tag, 66
commandline, 56
comment, 60, 66
comment_tag, 66
complete, 62, 66
configuration, 3
continue, 58, 66
copy_tag, 66
cycle, 42

D

database, 44
database_tag, 67
dependencies, 9
directory_structure, 67
do log, 46
do_tag, 68
dtd, 17

E

environment handling, 34
external files, 10
extract specifc block, 55

F

faq, 49
files, 10
fileset_tag, 69

G

general commandline options, 57
general_structure.xml, 69
general_structure.yaml, 70

H

hello world, 4
help, 7, 62

I

include, 27
include_tag, 71
include-path_tag, 71
info, 61, 71
input format, 4
installation, 3
introduction, 1
iofile_tag, 71
iteration, 39

J

jobssystem, 24
jube_pattern, 72
jube_variables, 72

L

link_tag, 73
loading files, 10
log, 61, 73
logging, 7

M

multiple benchmarks, 32

O

output, 60, 73

P

parallel, 43
parameter dependencies, 35
parameter groups, 51
parameter space creation, 8
parameter update, 37
parameter_space, 73
parameter_tag, 73
parameterset_tag, 74
pattern, 19
pattern_tag, 75
patternset_tag, 76
perl, 18, 19

platform independent, 31
prepare_tag, 76
python, 18, 19

R

remove, 60, 76
restart workpackage, 56
result, 12, 59, 77
result_tag, 77
run, 57, 77

S

schema validation, 17
scripting, 18, 19
selection_tag, 77
shared operations, 33
shell, 18, 19
statistic values, 22
statistical_values, 77
status, 62, 78
step dependencies, 9
step_tag, 78
sub_tag, 79
substituteset_tag, 79
substitution, 10
syslog_tag, 79

T

table, 12
table_tag, 80
tag, 80
tagging, 29, 80
tags_tag, 80
tutorial, 1
types, 81

U

update, 63, 81
update_mode, 81

V

validation, 17

W

workdir change, 52
workpackage, 81

X

XML character handling, 53

Y

YAML character handling, 53